



DEPARTMENT OF ENGINEERING CYBERNETICS

TTK4550

Specialization Project

Authors:

Anders Haarberg Eriksen, Øyvind Stavdahl & Devon Rohlf

Date: 24th May 2026

Abstract

This thesis investigates modelling, parameter estimation, and control strategies for a tendon-driven hand exoskeleton intended to assist hand function in spinal-cord-injured patients. The exoskeleton is actuated by brushed DC motors through Bowden-tube cable transmissions, introducing uncertain dynamics, friction, and limited sensing at the hand. The work focuses on a single finger as a representative subsystem of the full 6-DOF exoskeleton.

A planar three-link finger model was derived using the Euler-Lagrange equations, including joint stiffness and damping. A brushed DC motor model was also developed, and relevant motor parameters were estimated from datasheet values. For parameters not available from the datasheet, a gradient-based identification method was formulated and evaluated in simulation. The motor and finger models were then connected through a cable constraint, where the resulting Lagrange multipliers were interpreted as cable tensions.

A discrete-time current controller was designed using proportional feedback with a feedforward term to compensate for back-EMF. Simulations showed that the controller can track the armature current rapidly compared to the mechanical dynamics, supporting the use of armature current as an effective control input for torque and position control. For force control, simulations indicated that Bowden-tube friction causes steady-state force errors when only motor-side information is used, while integrated fingertip force sensors allow the desired applied force to be reached more reliably. For position control, a model reference adaptive control scheme was designed using the motor angle as a proxy for fingertip position. The adaptive controller improved tracking over time, but the adaptive gains were observed to grow slowly due to nonlinearities, saturation, and modelling assumptions. The results therefore suggest that the adaptive scheme is most suitable as an automatic tuning method for finding fixed controller gains.

Overall, the proposed controllers provide a simulation-based proof of concept for future implementation on the physical exoskeleton. Experimental validation is required to assess the effects of cable elasticity, Bowden-tube friction, whiffle-tree geometry, sensor limitations, and user-dependent hand dynamics.

Project Assignment

Candidate	Anders Haarberg Eriksen
Programme	Cybernetics and Robotics
Project title	Adaptive Control and Parameter Estimation for Motion Control in a Tendon-Driven Hand Exoskeleton
Norwegian title	Adaptiv regulering og parameterestimering for bevegelsesstyring i et senedrevet håndeksoskjelett
Supervisors	Øyvind Stavdahl, Devon Rolhf
Institution	Department of Engineering Cybernetics, NTNU, in collaboration with the N-squared Lab, FAU, Erlangen, Germany

Modern hand exoskeletons require precise and reliable control of the relevant mechanical states to enable natural, predictable, and safe interaction with the environment. However, the current implementation of the present glove–cable system introduces nonlinearities, uncertain mechanical parameters, and inconsistent finger positioning, making accurate control difficult. To address these challenges, an adaptive control framework is desired that can estimate system parameters in real time, compensate for nonlinearities, and ensure stable behavior during both free motion and object interaction.

The goal of this project is to design an adaptive controller for a 6-DOF hand exoskeleton, capable of regulating both torque and fingertip positioning. This includes identifying relevant mechanical parameters and creating a nonlinear model of the system. Additionally, modifications to the glove mechanics and integration of fingertip sensors could enable better positioning and torque estimation and facilitate switching between control modes and should therefore be investigated. The final system should allow reliable, intuitive control of finger motion while providing insight into system properties such as friction and internal forces.

The work should thus include the following:

- Development of a nonlinear model of a single finger to support adaptive control.
- Design of an adaptive control architecture for both torque output and finger positioning.
- Estimation of mechanical parameters and internal forces.
- Evaluation of fingertip sensors to support mode switching and refined control.
- Documentation and of the system architecture, experiments, and control performance.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Overview of the current setup	2
1.3	Outline of thesis	4
1.4	AI declarations	4
2	Theoretical background	5
2.1	Anatomy	5
2.1.1	Anatomical directional terms	5
2.1.2	Anatomical terms of movement	5
2.1.3	Hand anatomy	6
2.2	Modelling theory	9
2.2.1	Euler-Lagrange equations	9
2.2.2	Homogenous transformations	9
2.3	Control theory	10
2.3.1	Discrete-time system analysis	10
2.3.2	State-Space representation	11
2.3.3	Controllability	11
2.3.4	Static parametric model (SPM)	12
2.3.5	Gradient method for estimating unkown parameters	12
2.3.6	Model Reference Adaptive Control (MRAC)	13
3	Modelling	15
3.1	Finger model	15
3.1.1	Kinetic energy	17
3.1.2	Potential energy	19
3.1.3	Generalized forces	19
3.1.4	Rigid-body dynamics	19
3.2	Brushed DC Motor model	21
3.2.1	Estimating motor parameters	22
3.2.2	Estimating the armature inductance and rotor inertia	23

3.2.3	Mass-spring-damper load	26
3.2.4	State-space model of brushed DC motor with load	26
4	Controller design	27
4.1	Current controller	27
4.1.1	Discretization of current controller	28
4.1.2	Stability analysis and tuning of controller	28
4.1.3	Simulation of current controller	29
4.1.4	Evaluation of controller	32
4.2	Torque control	32
4.2.1	Force control with integrated fingertip force sensors	34
4.3	Model reference adaptive control (MRAC) for position control	36
4.3.1	Controllability of the system	36
4.3.2	MRAC implementation	37
4.3.3	Connecting the motors to the finger through a cable	38
4.3.4	Simulation of MRAC scheme with updated dynamics	40
4.3.5	Evaluation of MRAC scheme	42
5	Discussion	46
5.1	Current control	46
5.2	Torque and force control	47
5.3	Position controller using MRAC	48
5.4	Model assumptions and limitations	50
5.5	Practical implementation considerations	51
5.6	Future work	52
6	Conclusion	55

List of Figures

1.1	Setup for one motor controlling a finger.	2
1.2	Image of the system without the cable, bowden tube, whiffle tree or glove. Image taken by Devon Rohlf.	3
2.1	The anatomical position, with the corresponding directional terms. Image from TeachMeAnatomy, 2026a	6
2.2	Anatomical terms of movement. Image from TeachMeAnatomy, 2026b	6
2.3	The bones of the hand. Image from author.	7
2.4	The joints of the hand. Image from author.	8
3.1	Visualization of the finger model, with three revolute joints, where the relative joint angles, and the lengths of each phalanx are labeled.	16
3.2	Circuit diagram of armature circuit of the DC motor model, with output torque and angular position.	21
3.3	Parameter identification of the armature inductance L_a and rotor inertia J_m using a step input in the armature voltage E_a	25
4.1	Armature current step response for discrete-time P+FF and P controllers tracking a 1 A reference, with $T_s = 0.2$ ms and initial motor speed $\omega_0 = 5$ rad s ⁻¹	30
4.2	Armature current step response for discrete-time P+FF and P controllers tracking a 1 A reference, with $T_s = 0.2$ ms and initial motor speed $\omega_0 = 0$ rad s ⁻¹	30
4.3	Effect of current measurement noise on the armature current response using a discrete-time P+FF controller with $T_s = 0.2$ ms.	31
4.4	Effect of current measurement noise on the armature current response using a discrete-time P+FF controller with $T_s = 1.0$ ms.	31
4.5	Torque control with a P controller when gripping a rigid object.	34
4.6	Force control with friction. With and without force sensors in fingertips. Both controllers have $T_s = 0.2$ ms.	35

4.7	Force application on the finger. The flexion motor applies a force F_f on the flexion cable, which gets distributed to the three phalanges through the whiffle tree. Similarly, the extension motor applies a force F_e on the extension cable, which also gets distributed to the three phalanges through its own whiffle tree.	41
4.8	Simulation results of the MRAC scheme for flexion and extension motor over 10 s.	43
4.9	Simulation results of the MRAC scheme for flexion and extension motor after 90 s.	43
4.10	Adaptive gains and error over time for MRAC scheme for 100 s, where K_1 and L keep growing together.	44
4.11	Simulation results of the MRAC scheme for flexion and extension motor over 10 s, with the adaptive gains found in Figure 4.10 after 100 s, but with the gains fixed instead of adaptive.	45

List of Tables

3.1	Parameters used in the finger model.	17
3.2	Constants in system matrices.	21
3.3	Relevant motor parameters extracted from the datasheet of the motor.	23
4.1	Physical parameters used for the finger model.	42

Chapter 1

Introduction

This thesis presents the development and simulation-based validation of control strategies for fingertip positioning and force control for a 6-DOF tendon-driven hand exoskeleton. The exoskeleton is developed by the N-Squared lab at Friedrich-Alexander Universität Erlangen-Nürnberg (FAU), Erlangen, Germany, and is intended for use in spinal-cord-injured (SCI) patients, with the goal of improving hand function and independence.

1.1 Motivation

SCI-patients may end up having the ability to control their muscles, reduced or even lost due to their nerves being damaged. They may retain their limbs, but are unable to control them at will. This differs from people that are born with missing limbs, or that through life lose their limbs. In these cases, total reconstruction of the limbs, like with prosthetics are needed to restore movement. Instead of prosthetics, people with muscular dysfunction will need something to move their still existing limbs for them. This is usually done by an orthosis, also referred to as an exoskeleton.

Research has shown that some SCI-patients with muscular dysfunction, still retain minimal motor neuron activity in their forearm (Ting et al., 2019). This activity can be measured with intramuscular electromyography (iEMG) (‘ICS Glossary’, 2026), and classifiers or regression models can then be used on this data to interpret the user’s intent. With the intent of the user, an external exoskeleton could then be used to move the fingers in accordance to what the user intended, regaining the users independence and improving their quality of life.

The human hand, with all its dexterity and versatility, makes it capable of performing a wide range of tasks, from delicate manipulations to powerful grips. The loss of such a vital tool can be devastating. Therefore, regaining even a fraction of its functionality can have a profound impact on the lives of those affected. In attempt to restore as much of the hand’s functionality as possible, the N-Squared lab is hence developing a 6-DOF tendon-driven hand exoskeleton; 1-DOF for each of the five fingers, and an

additional 1-DOF for the thumb's abduction and adduction. The interpretation of the user's intent is handled by someone else on the team, while the focus of this thesis is on the control of the exoskeleton itself. It will be assumed that the user's intent, and thereby the reference signals for the controllers, are accurately decoded.

1.2 Overview of the current setup

In the current setup, each degree of freedom (DOF) of the exoskeleton is actuated by two brushed DC motors, one for flexion and one for extension. A spindle is mounted on each motor, to which a cable is connected. The cable is connected radially to the axis of rotation with a radius r_{spindle} . The cable is routed over a pulley that's mounted on a force sensor. Further it's routed through a bowden tube, and finally attached to a glove mounted on the users hand.

When the motor spins, the cable tightens and pulls on the finger, similar to the tendons of the hand, hence the name, tendon-driven hand exoskeleton. To spread the force on the finger, and ideally facilitate for a more natural movement, a whiffle tree is considered between the cable and the finger. The exact configuration and design is still being worked on. The idea is that it will spread the force from one cable to three cables going to different points on the finger. The exact attachment points are also still being worked on, but the idea is to have discs mounted on the sides of each phalanx, to which the cables connect to. The cables going from the whiffle trees will be routed on the backside of the hand, not to interfere with the user's grasping. An estimate of the resulting attachment points of the cables from the flexion and extension motor on the finger is shown later in Figure 4.7. The setup for one motor controlling one finger is shown in Figure 1.1. The hardware of the system includes brushed DC motors, motor drivers, an Arduino Giga, magnetic encoders for measuring the angle of the motors, analog force sensors and corresponding amplifiers, and soon hall effect current sensors for measuring the current going to the motors. A picture of the system without the cables, bowden tubes, whiffle trees or glove is shown in Figure 1.2.

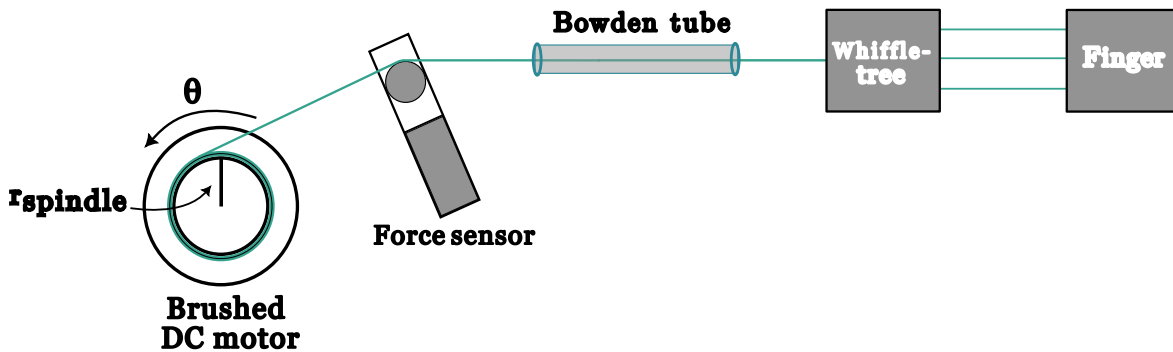


Figure 1.1: Setup for one motor controlling a finger.

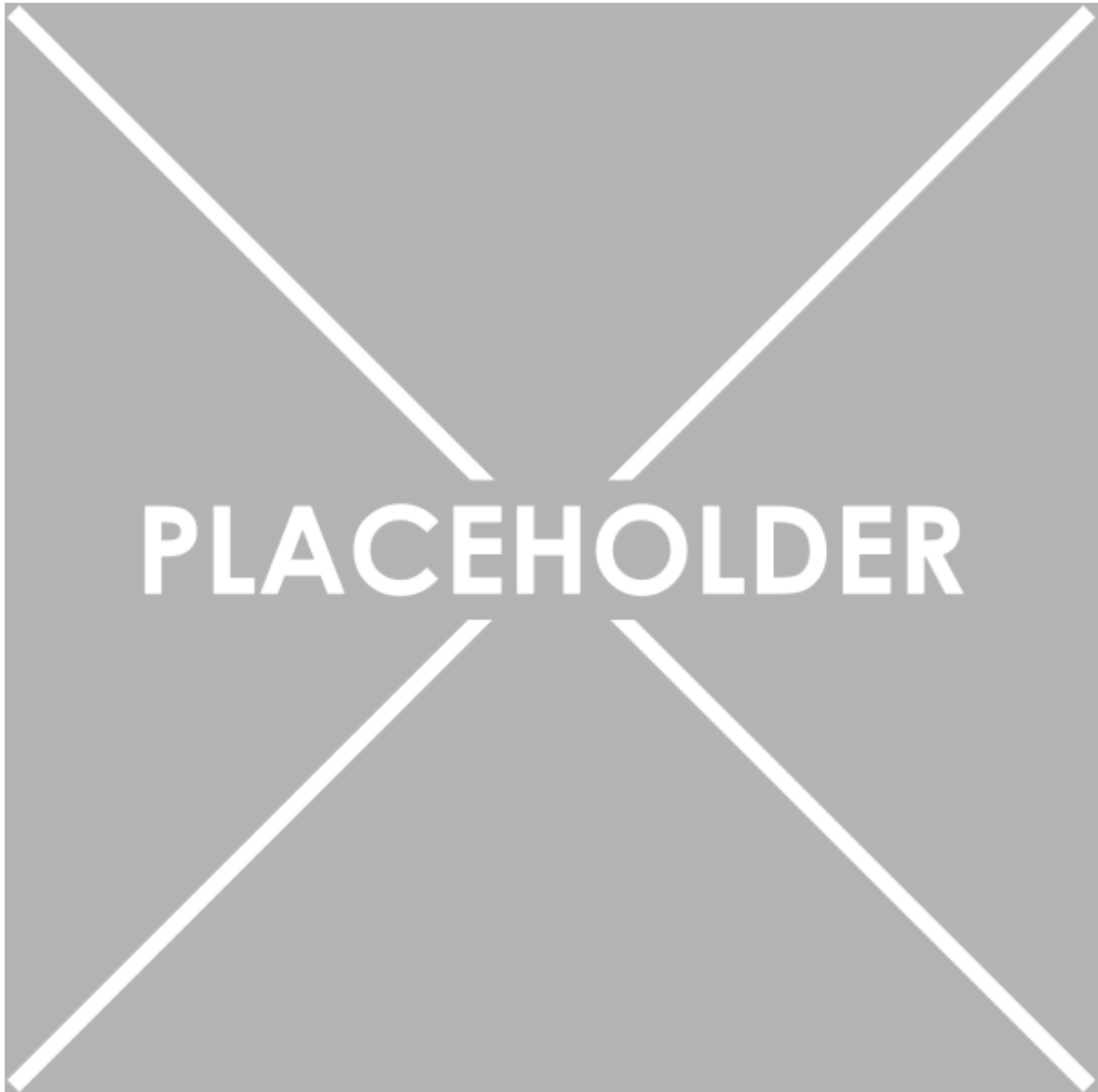


Figure 1.2: Image of the system without the cable, bowden tube, whiffle tree or glove. Image taken by Devon Rohlf.

1.3 Outline of thesis

The thesis is structured as follows: Chapter 2 provides the theoretical background necessary for understanding the subsequent chapters, including anatomy, modelling, and control theory. Chapter 3 details the modelling of the finger dynamics using the Euler-Lagrange equations. Further the chapter covers the dynamics of the brushed DC motors, and estimates of the model parameters. The chapter also includes Chapter 4 presents the design, analysis and simulation-based validation of a current controller, torque and force controller, and position control for controlling the position of the fingertip. Chapter 5 discusses the results and findings of the thesis, and provides an outlook on future work, and finally, Chapter 6 concludes the thesis.

1.4 AI declarations

LLMs have been used to assist in development of this thesis. Claude Code and ChatGPT have been used for several purposes, including assisting in coding, debugging, and proofreading this thesis. However, the text, methods developed, and figures in this thesis are all human-made. All contributions by an LLM have been reviewed and approved line by line.

Chapter 2

Theoretical background

2.1 Anatomy

The hand is a complex biomechanical system, comprised of many different bones, muscles, tendons, and ligaments. To explain different aspects of the hand in relation to one another, we use *anatomical terminology* [National Cancer Institute, 2026], or more specifically *anatomical directional terms*. Additionally when describing actions of muscles upon the skeleton, we use *anatomical terms of movement* [TeachMeAnatomy, 2026b].

2.1.1 Anatomical directional terms

Anatomical directional terms are used in relation to the *anatomical position*, shown in Figure 2.1, where the body is assumed standing upright, with arms at the sides and palms facing forward. Relevant terms include: *proximal*, *distal*.

- **Proximal** means toward or nearest the point of origin of a part.
- **Distal** means away from or farthest from the point of origin of a part.

2.1.2 Anatomical terms of movement

To explain the actions of muscles upon the skeleton, we use anatomical terms of movement, which describes the movement of a body part in relation to the anatomical position. Relevant terms include: *flexion*, *extension*, *abduction*, *adduction*.

- **Flexion** is a movement that decreases the angle between two body parts.
- **Extension** is a movement that increases the angle between two body parts.
- **Abduction** is a movement away from the midline of the body.

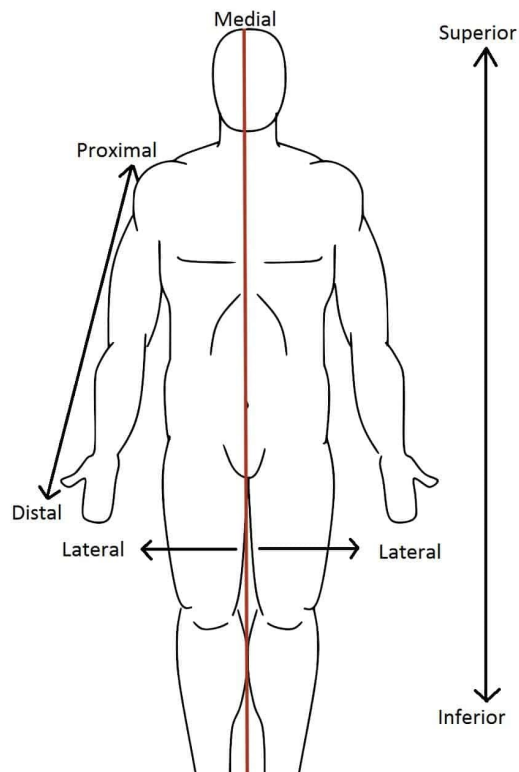


Figure 2.1: The anatomical position, with the corresponding directional terms. Image from TeachMeAnatomy, [2026a](#).

- **Adduction** is a movement toward the midline of the body.

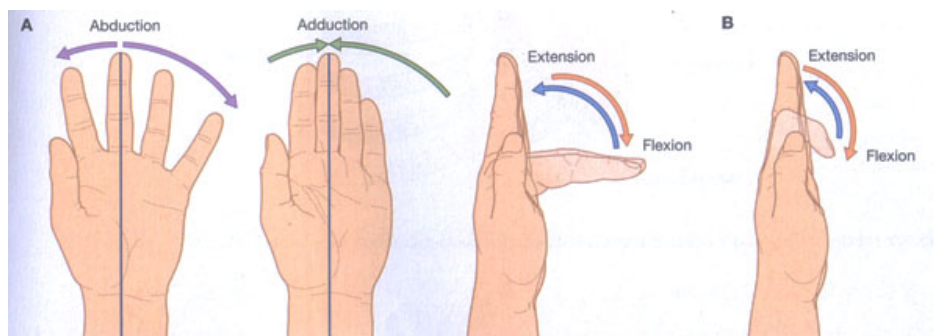


Figure 2.2: Anatomical terms of movement. Image from TeachMeAnatomy, [2026b](#).

2.1.3 Hand anatomy

The hand is comprised of several bones, muscles, tendons, and ligaments. The bones, illustrated in Figure 2.3, include the *carpals*, *metacarpals*, and *phalanges*. The carpals make out the wrist, the metacarpals make out the palm, and the phalanges make out the fingers. The thumb has two phalanges, while the other fingers have three. The bones of interest are the phalanges (marked in green), as they are the ones being moved by the exoskeleton. The metacarpals are assumed stationary for the task at hand, as the exoskeleton will provide stiffness to the palm and wrist.

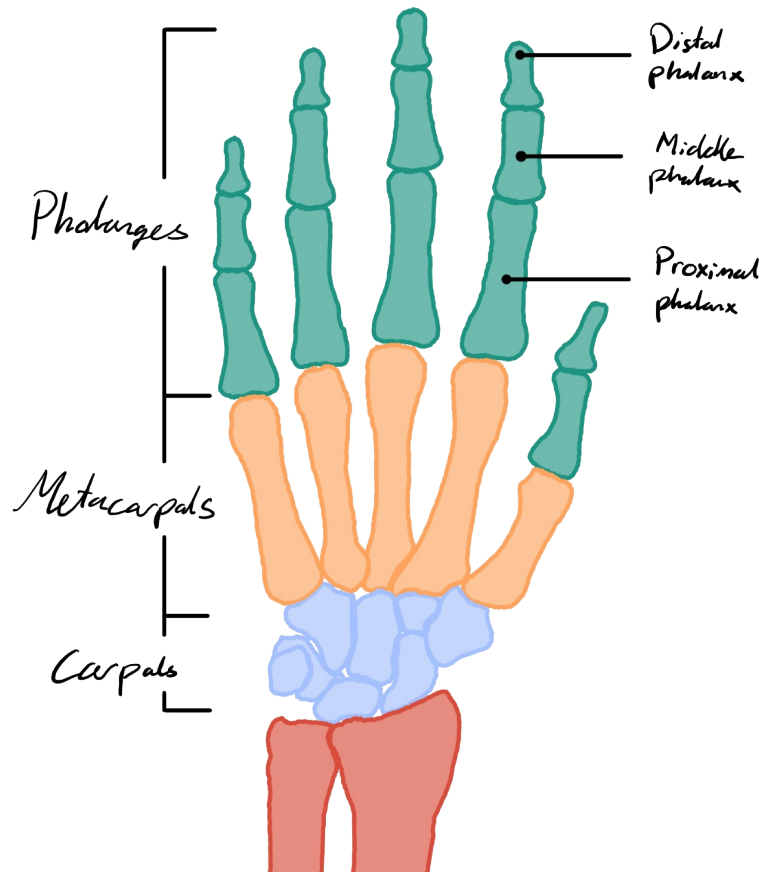


Figure 2.3: The bones of the hand. Image from author.

The phalanges are further divided into three sections: the *proximal phalanges* (PP), the *middle phalanges* (MP), and the *distal phalanges* (DP). The proximal phalanges are the ones closest to the palm, while the distal phalanges are the ones furthest from the palm. The middle phalanges are the ones in between. The thumb does not have a middle phalanx, as it only has two phalanges.

Further details about the joints are shown in Figure 2.4. The joints of interest are the *metacarpophalangeal joints* (MCP), the *proximal interphalangeal joints* (PIP), and the *distal interphalangeal joints* (DIP). The MCP joints are the ones between the metacarpals and the proximal phalanges, while the PIP joints are the ones between the proximal phalanges and the middle phalanges, and the DIP joints are the ones between the middle phalanges and the distal phalanges. The thumb only has an MCP joint and an interphalangeal joint (IP), as it does not have a middle phalanx.

At each of the joints, there are collateral ligaments that provide stability. For the PIP, IP and DIP joints, they provide stiffness that prevents side to side motion. For the MCP joints they're relaxed when the finger is extended, facilitating for adduction and abduction, but stiff when the finger is flexed.

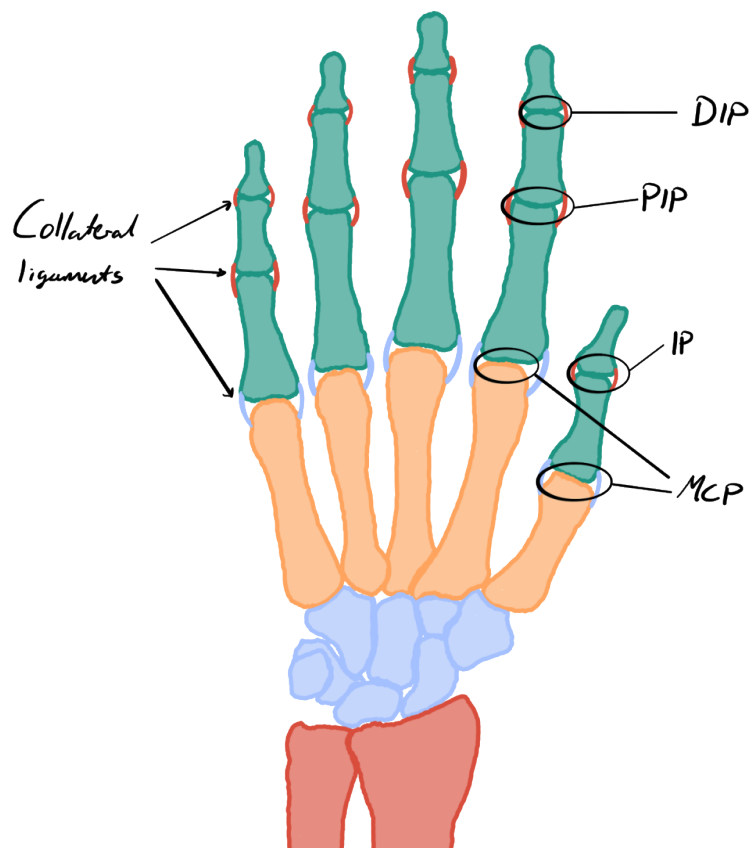


Figure 2.4: The joints of the hand. Image from author.

2.2 Modelling theory

2.2.1 Euler-Lagrange equations

The Euler-Lagrange equations are a set of second-order ordinary differential equations that describe the dynamics of a system. They are derived using the total potential and kinetic energy of the system, denoted by V and T . Together, they form the Lagrangian of the system, denoted by $\mathcal{L}$, which is defined as:

$$\mathcal{L} = T - V. \quad (2.1)$$

The Euler-Lagrange equations in vector form are given by:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \mathbf{Q}, \quad (2.2)$$

where $\mathbf{q}$ are the generalized coordinates, which describe the configuration of the system, and $\mathbf{Q}$ are the generalized forces, describing the external influences on the system.

2.2.2 Homogenous transformations

When modelling kinematic chains, we often need to move between frames. This often includes both rotation and translation. An efficient way to represent this is by using *homogenous transformations*.

Definition

For a 2D system, the rotation matrix for a counter-clockwise rotation by an angle θ is given by:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}. \quad (2.3)$$

Given a translation vector from frame i to frame j , written in terms of frame i , $\mathbf{d}_i^j = \begin{bmatrix} d_x^i \\ d_y^i \end{bmatrix}$, the homogenous transformation matrix from frame j to frame i , describing the rotation θ and translation $\mathbf{d}_i^j$ from frame i to frame j , can be expressed as:

$$\mathbf{H}_j^i = \begin{bmatrix} \mathbf{R}(\theta) & \mathbf{d}_i^j \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix}. \quad (2.4)$$

Points represented in terms of homogenous transformations are represented as a 3D vector, where the first two elements represent the coordinates of the point in the plane,

and the third element is always 1. For example, a point $\mathbf{p}_n$ in frame n with coordinates (x, y) can be represented as:

$$\mathbf{p}_n = \begin{bmatrix} p_x^n \\ p_y^n \\ 1 \end{bmatrix}. \quad (2.5)$$

Useful properties

Given a kinematic chain of size n , we can express the homogenous transformation from frame n to frame 1 as the product of the homogenous transformations between each consecutive frame:

$$\mathbf{H}_n^1 = \mathbf{H}_2^1 \mathbf{H}_3^2 \cdots \mathbf{H}_n^{n-1}. \quad (2.6)$$

Given a point $\mathbf{p}_n$ in frame n , we can express it in frame 1 as:

$$\mathbf{p}_1 = \begin{bmatrix} p_x^1 \\ p_y^1 \\ 1 \end{bmatrix} = \mathbf{H}_2^1 \mathbf{H}_3^2 \cdots \mathbf{H}_n^{n-1} \mathbf{p}_n = \mathbf{H}_n^1 \mathbf{p}_n. \quad (2.7)$$

2.3 Control theory

2.3.1 Discrete-time system analysis

When realizing a controller on a digital computer, the system is often modelled as a discrete-time system, as the samples are taken at discrete time steps. A sample of state y at time step k is denoted as $y[k]$. The sampling time, denoted by T_s , is the time between two consecutive samples, such that if $y[k] = y(t)$, then $y[k+1] = y(t+T_s)$. Usually the input of the system is denoted $u[k]$. Often the discrete-system can be expressed as a *auto-regressive* (AR) model, where the current state of the system is given by a linear combination of the previous states and inputs, as shown in the following equation:

$$y[k] = \sum_{i=1}^n a_i y[k-i] + \sum_{j=1}^m b_j u[k-j], \quad (2.8)$$

To analyze the stability of a discrete-time system, we can use the *z-transform*.

Z-transform

The z-transform of a discrete-time signal $y[k]$ is defined as:

$$y(z) = \sum_{k=0}^{\infty} y[k]z^{-k}. \quad (2.9)$$

Some useful properties of the z -transform include:

- Linearity: $af[k] + bg[k] \xrightarrow{\mathcal{Z}} af(z) + bg(z)$.
- Time-shift: $y[k - n] \xrightarrow{\mathcal{Z}} z^{-n}y(z)$.

Stability analysis of AR model

Using the properties of the z -transform, the AR model can be written in the z -domain as:

$$h(z) \triangleq \frac{y(z)}{u(z)} = \frac{\sum_{j=1}^m b_j z^{-j}}{1 - \sum_{i=1}^n a_i z^{-i}}, \quad (2.10)$$

where $h(z)$ denotes the system transfer function. For a causal discrete-time system, stability requires all poles of $h(z)$ to lie strictly inside the unit circle, i.e.,

$$|p_i| < 1, \quad i = 1, \dots, n. \quad (2.11)$$

2.3.2 State-Space representation

A way to express a n 'th order ordinary differential equation (ODE) is to express it as a system of n first order ODEs. This is called the *state-space representation* of the system. A state-space representation commonly used in control theory for linear time-invariant (LTI) systems is the following:

$$\begin{aligned} \dot{\mathbf{x}}(t) &= \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{d}(t), & \mathbf{x}(t) &\in \mathbb{R}^n, & \mathbf{u}(t) &\in \mathbb{R}^m, & \mathbf{y}(t) &\in \mathbb{R}^p, & \mathbf{d}(t) &\in \mathbb{R}^n, \\ \mathbf{y}(t) &= \mathbf{C}\mathbf{x}(t), & \mathbf{A} &\in \mathbb{R}^{n \times n}, & \mathbf{B} &\in \mathbb{R}^{n \times m}, & \mathbf{C} &\in \mathbb{R}^{p \times n}. \end{aligned} \quad (2.12)$$

Here, $\mathbf{x}(t)$ is the state vector, which contains all the information about the system at time t . $\mathbf{u}(t)$ is the input vector at time t . $\mathbf{y}(t)$ is the output vector, which contains all the measurements of the system at time t . $\mathbf{d}(t)$ is a disturbance vector.

2.3.3 Controllability

A system is said to be *controllable* if it is possible to drive the state of the system from any initial state to any desired final state in finite time, using an appropriate control input. For a LTI system, controllability can be determined by the rank of the controllability matrix $\mathcal{C}$, defined as:

$$\mathcal{C} = [\mathbf{B}, \mathbf{A}\mathbf{B}, \mathbf{A}^2\mathbf{B}, \dots, \mathbf{A}^{n-1}\mathbf{B}]. \quad (2.13)$$

If $\text{rank}(\mathcal{C}) = n$, where n is the dimension of the state vector $\mathbf{x}$, then the system is controllable.

2.3.4 Static parametric model (SPM)

A *static parametric model* (SPM) is a special form of a dynamical system relating the input and output of a system when there are unknown parameters. P. Ioannou and Fidan, 2006, p. 13 defines a SPM as follows:

$$z = \boldsymbol{\theta}^{*\top} \boldsymbol{\phi}, \quad (2.14)$$

where $z \in \mathbb{R}$, $\boldsymbol{\phi} \in \mathbb{R}^n$ are available for measurement, while $\boldsymbol{\theta}^* \in \mathbb{R}^n$ is the unknown parameter vector.

2.3.5 Gradient method for estimating unknown parameters

Given a SPM, we can use the *gradient method* to estimate the unknown parameters. P. A. Ioannou and Sun, 2012, p. 180–185 defines the gradient method as follows. Given the SPM in Equation (2.14), we can define the estimate of z , $\hat{z}$, as:

$$\hat{z} = \boldsymbol{\theta}^\top \boldsymbol{\phi}, \quad (2.15)$$

where $\boldsymbol{\theta} \in \mathbb{R}^n$ is the estimated parameter vector. We can define the estimation error as:

$$\epsilon = \frac{z - \hat{z}}{m_s^2}, \quad (2.16)$$

where m_s^2 is defined such that ϕ/m_s is bounded. Defining the adaptive law as:

$$\dot{\boldsymbol{\theta}} = \boldsymbol{\Gamma} \boldsymbol{\phi} \epsilon, \quad \boldsymbol{\Gamma} = \boldsymbol{\Gamma}^\top > 0, \quad (2.17)$$

P. A. Ioannou and Sun, 2012 proves that $\boldsymbol{\theta} \rightarrow \boldsymbol{\theta}^*$ as $t \rightarrow \infty$, given that $\boldsymbol{\phi}$ is persistently exciting (PE). A signal being PE means that there exists a $T > 0$ and $\alpha > 0$ such that:

$$\int_t^{t+T} \boldsymbol{\phi}(\tau) \boldsymbol{\phi}^\top(\tau) d\tau \geq \alpha \mathbf{I}, \quad \forall t \geq 0. \quad (2.18)$$

Essentially, this means that $\boldsymbol{\phi}$ must be sufficiently rich in frequency content, and cannot be a constant signal.

2.3.6 Model Reference Adaptive Control (MRAC)

Model Reference Adaptive Control (MRAC) is a control strategy that aims to make the output of a system follow the output of a reference model, even in the presence of uncertainties or disturbances. The following definition of MRAC is based on P. A. Ioannou and Sun (2012, pp. 325–327).

Given the n th order LTI-system in state-space form,

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t), \quad \mathbf{x}(t) \in \mathbb{R}^n, \mathbf{u}(t) \in \mathbb{R}^m, \quad (2.19)$$

and the reference model,

$$\frac{d}{dt}\mathbf{x}_m(t) = \mathbf{A}_m\mathbf{x}_m(t) + \mathbf{B}_m\mathbf{r}(t), \quad \mathbf{x}_m(t) \in \mathbb{R}^n, \mathbf{r}(t) \in \mathbb{R}^m. \quad (2.20)$$

The reference model and input $\mathbf{r}(t)$ are chosen so that $\mathbf{x}_m(t)$ represents the desired trajectory that $\mathbf{x}(t)$ should follow. Crucially, $\mathbf{A}_m$ must be a stable matrix, and $(\mathbf{A}, \mathbf{B})$ must be *controllable* for this to work. If matrices $\mathbf{A}$ and $\mathbf{B}$ are known, then the control input $\mathbf{u}(t)$ can be designed as follows:

$$\mathbf{u}(t) = -\mathbf{K}^*\mathbf{x}(t) + \mathbf{L}^*\mathbf{r}(t), \quad \mathbf{K}^* \in \mathbb{R}^{m \times n}, \mathbf{L}^* \in \mathbb{R}^{m \times m}. \quad (2.21)$$

This obtains the closed loop system:

$$\frac{d}{dt}\mathbf{x}(t) = (\mathbf{A} - \mathbf{B}\mathbf{K}^*)\mathbf{x}(t) + \mathbf{B}\mathbf{L}^*\mathbf{r}(t). \quad (2.22)$$

If we choose $\mathbf{K}^*$ and $\mathbf{L}^*$ such that,

$$\mathbf{A} - \mathbf{B}\mathbf{K}^* = \mathbf{A}_m, \quad \mathbf{B}\mathbf{L}^* = \mathbf{B}_m, \quad (2.23)$$

then the closed loop system becomes:

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{A}_m\mathbf{x}(t) + \mathbf{B}_m\mathbf{r}(t). \quad (2.24)$$

Hence the closed-loop dynamics of $\mathbf{x}(t)$ is the same to Equation (2.20), and $\mathbf{x}(t) \rightarrow \mathbf{x}_m(t)$ exponentially fast. But for most cases, $\mathbf{A}$ and $\mathbf{B}$ are not known, which means that $\mathbf{K}^*$ and $\mathbf{L}^*$ has to be estimated online. Assuming that $\mathbf{K}^*$ and $\mathbf{L}^*$ exist, we'll change the input in Equation (2.21) to:

$$\mathbf{u}(t) = -\mathbf{K}(t)\mathbf{x}(t) + \mathbf{L}(t)\mathbf{r}(t), \quad \mathbf{K}(t) \in \mathbb{R}^{m \times n}, \mathbf{L}(t) \in \mathbb{R}^{m \times m}. \quad (2.25)$$

Adaptive Law

Defining the tracking error $\mathbf{e}(t)$ as:

$$\mathbf{e}(t) = \mathbf{x}(t) - \mathbf{x}_m(t). \quad (2.26)$$

Solving for $\mathbf{P}$ for a $\mathbf{Q} = \mathbf{Q}^\top > 0$ in the Lyapunov equation:

$$\mathbf{A}_m^\top \mathbf{P} + \mathbf{P} \mathbf{A}_m = -\mathbf{Q}, \quad (2.27)$$

we have the adaptive law:

$$\begin{aligned} \dot{\mathbf{K}}(t) &= \mathbf{B}_m^\top \mathbf{P} \mathbf{e} \mathbf{x}^\top \text{sgn}(l), \\ \dot{\mathbf{L}}(t) &= -\mathbf{B}_m^\top \mathbf{P} \mathbf{e} \mathbf{r}^\top \text{sgn}(l). \end{aligned} \quad (2.28)$$

P. A. Ioannou and Sun, [2012](#) proves, that $\mathbf{K}(t)$, $\mathbf{L}(t)$, and $\mathbf{e}(t)$ are bounded, and the tracking error $\mathbf{e}(t) \rightarrow 0$ as $t \rightarrow \infty$.

Chapter 3

Modelling

3.1 Finger model

For modelling the finger, we're assuming that the motion is limited to a 2D-plane. This is mostly true, due to the collateral ligaments at the DIP- and the PIP-joint, preventing excessive side-to-side bending, as discussed in Section 2.1.3. The MCP joint doesn't have the same support-structure, so it's free to move side to side, but this isn't as relevant to model for the task at hand, as the lateral movement from the cable pulling on the finger is minimal.

For our model, we assume that the finger is composed of three links, with three revolute joints, like a *planar manipulator* ('Planar Manipulator - an Overview | ScienceDirect Topics', 2026). To model the stiffness from the tendons and ligaments, we'll add a torsional spring at each joint. The springs will have a non-zero equilibrium angle, representing the natural resting position of the finger joints. The damping from the soft tissue will be modeled by torsional dampers. A schematic of the model can be seen in Figure 3.1. We'll model the links as rigid cylinders, with uniform mass distribution, and the joints as ideal revolute joints, with no friction. The parameters of the model are listed in Table 3.1.

To model the dynamics of the finger, we'll use the Euler-Lagrange equations as shown in Section 2.2.1. We'll define our generalized coordinates as the relative joint angles, φ_{MCP} , φ_{PIP} and φ_{DIP} :

$$\mathbf{q} \triangleq \begin{bmatrix} \varphi_{\text{MCP}} \\ \varphi_{\text{PIP}} \\ \varphi_{\text{DIP}} \end{bmatrix} \implies \dot{\mathbf{q}} = \begin{bmatrix} \dot{\varphi}_{\text{MCP}} \\ \dot{\varphi}_{\text{PIP}} \\ \dot{\varphi}_{\text{DIP}} \end{bmatrix} \implies \ddot{\mathbf{q}} = \begin{bmatrix} \ddot{\varphi}_{\text{MCP}} \\ \ddot{\varphi}_{\text{PIP}} \\ \ddot{\varphi}_{\text{DIP}} \end{bmatrix}. \quad (3.1)$$

To derive the equations of motion, we need to define the kinetic energy, T , and the potential energy, V of our system.

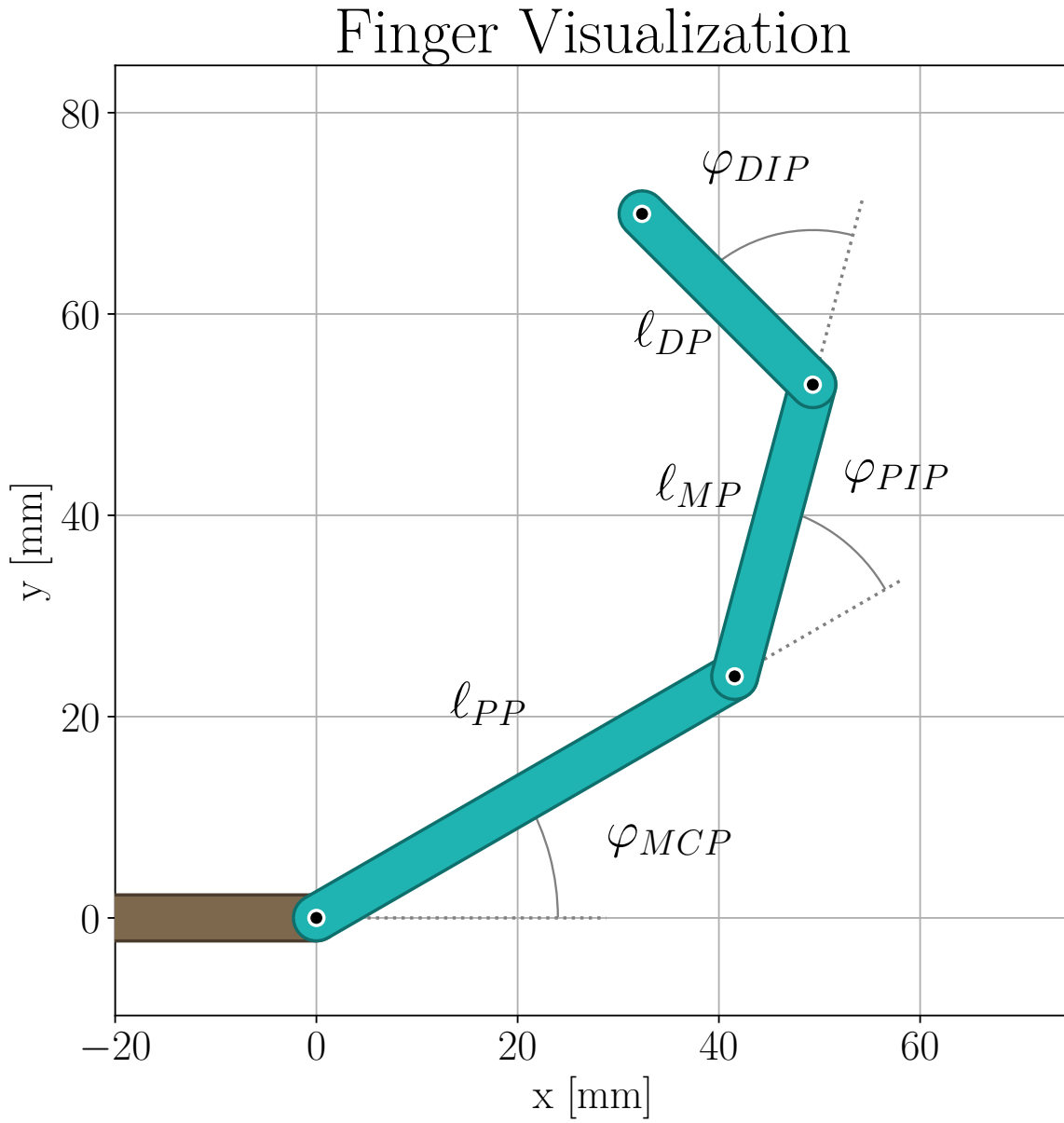


Figure 3.1: Visualization of the finger model, with three revolute joints, where the relative joint angles, and the lengths of each phalanx are labeled.

Table 3.1: Parameters used in the finger model.

Parameter	Link/joint	Symbol	Value	Unit
Mass	PP	m_{PP}	20	g
	MP	m_{MP}	15	
	DP	m_{DP}	20	
Length	PP	l_{PP}	48	mm
	MP	l_{MP}	30	
	DP	l_{DP}	24	
Radius	PP	r_{PP}	8.5	mm
	MP	r_{MP}	8.5	
	DP	r_{DP}	8.5	
Equilibrium angle	MCP	$\varphi_{MCP,eq}$	$\pi/6$	rad
	PIP	$\varphi_{PIP,eq}$	$\pi/4$	
	DIP	$\varphi_{DIP,eq}$	$\pi/12$	
Stiffness coefficient	MCP	k_{MCP}	0.5	N m rad ⁻¹
	PIP	k_{PIP}	0.5	
	DIP	k_{DIP}	0.5	
Damping coefficient	MCP	b_{MCP}	0.05	N m s rad ⁻¹
	PIP	b_{PIP}	0.05	
	DIP	b_{DIP}	0.05	

3.1.1 Kinetic energy

The kinetic energy will be the sum of the kinetic energy of each link, which can be decomposed into the rotational and translational kinetic energy, defined as:

$$T_{\text{tot}} = T_{\text{rot}} + T_{\text{trans}}. \quad (3.2)$$

Rotational kinetic energy

The rotational kinetic energy of the system is rather straightforward to derive, as the angular velocity of each link is equal to the time derivative of the joint angle. Since the angles are relative, the total angular velocity of each link is the sum of the time derivatives of the joint angles up to that link. Hence, the rotational kinetic energy can be expressed as:

$$T_{\text{rot}} = \frac{1}{2} J_{MCP} \dot{\varphi}_{MCP}^2 + \frac{1}{2} J_{PIP} (\dot{\varphi}_{MCP} + \dot{\varphi}_{PIP})^2 + \frac{1}{2} J_{DIP} (\dot{\varphi}_{MCP} + \dot{\varphi}_{PIP} + \dot{\varphi}_{DIP})^2. \quad (3.3)$$

Translational kinetic energy

The translational kinetic energy is a bit more complex to derive, as the linear velocity of each link depends on the geometry of the system. The linear velocity of each link can be derived by finding the position of the center of mass of each link and taking the time derivative. To find the position of the center of mass, we'll use homogenous transformations, as described in Section 2.2.2.

Let's start by defining frames for moving between the links. We'll define the static world frame as frame i , attached to the MCP joint, where the x-axis is alligned with the metacarpal. The next is frame PP , attached to the PIP joint, where the x-axis is alligned with the proximal phalanx. Subsequently we have frame MP , attached to the DIP joint, where the x-axis is alligned with the middle phalanx. Finally, we have frame DP , attached to the tip of the finger, where the x-axis is alligned with the distal phalanx. The homogenous transformations between these frames can be expressed as:

$$\begin{aligned} \mathbf{H}_{PP}^i &= \begin{bmatrix} \mathbf{R}(\varphi_{\text{MCP}}) & \mathbf{R}(\varphi_{\text{MCP}})\mathbf{e}_1 l_{\text{PP}} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix}, \\ \mathbf{H}_{MP}^{PP} &= \begin{bmatrix} \mathbf{R}(\varphi_{\text{PIP}}) & \mathbf{R}(\varphi_{\text{PIP}})\mathbf{e}_1 l_{\text{MP}} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix}, \\ \mathbf{H}_{DP}^{MP} &= \begin{bmatrix} \mathbf{R}(\varphi_{\text{DIP}}) & \mathbf{R}(\varphi_{\text{DIP}})\mathbf{e}_1 l_{\text{DP}} \\ \mathbf{0}_{1 \times 2} & 1 \end{bmatrix}. \end{aligned} \quad (3.4)$$

Assuming the the center of mass of each link is located at the midpoint of the link, we can express the position of the center of mass of each link in the world frame as:

$$\begin{aligned} \mathbf{p}_{\text{PP,CM}}^i &= \mathbf{H}_{PP}^i \begin{bmatrix} -l_{\text{PP}}/2 \\ 0 \\ 1 \end{bmatrix}, \\ \mathbf{p}_{\text{MP,CM}}^i &= \mathbf{H}_{PP}^i \mathbf{H}_{MP}^{PP} \begin{bmatrix} -l_{\text{MP}}/2 \\ 0 \\ 1 \end{bmatrix}, \\ \mathbf{p}_{\text{DP,CM}}^i &= \mathbf{H}_{PP}^i \mathbf{H}_{MP}^{PP} \mathbf{H}_{DP}^{MP} \begin{bmatrix} -l_{\text{DP}}/2 \\ 0 \\ 1 \end{bmatrix}. \end{aligned} \quad (3.5)$$

Extracting the x and y components of the position, defining them as $\mathbf{r}_{\text{PP,CM}}$, $\mathbf{r}_{\text{MP,CM}}$, and $\mathbf{r}_{\text{DP,CM}}$, we can take the time derivative to find the linear velocity of each center of mass, which can then be used to calculate the translational kinetic energy as:

$$T_{\text{trans}} = \frac{1}{2} \left[m_{\text{PP}} \dot{\mathbf{r}}_{\text{PP,CM}}^T \dot{\mathbf{r}}_{\text{PP,CM}} + m_{\text{MP}} \dot{\mathbf{r}}_{\text{MP,CM}}^T \dot{\mathbf{r}}_{\text{MP,CM}} + m_{\text{DP}} \dot{\mathbf{r}}_{\text{DP,CM}}^T \dot{\mathbf{r}}_{\text{DP,CM}} \right]. \quad (3.6)$$

3.1.2 Potential energy

For the potential energy one would normally consider the gravitational potential energy. However, since the finger is relatively light and the motion during gripping tasks is mostly in the horizontal plane, we'll omit the gravitational potential energy for simplicity. Also, for clarity we'll model the torsional springs as generalized forces, instead of including them in the potential energy. Hence, the potential energy of the system can be expressed as:

$$V = 0. \quad (3.7)$$

3.1.3 Generalized forces

The generalized forces in our system will be the torques from the torsional springs and dampers, as well as the torques from the cable pulling on the finger, and other external torques when the finger interacts with objects. The torques from the torsional springs can be expressed as:

$$\begin{aligned} \boldsymbol{\tau}_{\text{spring}} &= \begin{bmatrix} k_{\text{MCP}}(\varphi_{\text{MCP}} - \varphi_{\text{MCP,eq}}) \\ k_{\text{PIP}}(\varphi_{\text{PIP}} - \varphi_{\text{PIP,eq}}) \\ k_{\text{DIP}}(\varphi_{\text{DIP}} - \varphi_{\text{DIP,eq}}) \end{bmatrix} = \mathbf{K}(\mathbf{q} - \mathbf{q}_{\text{eq}}) \\ \mathbf{K} &\triangleq \begin{bmatrix} k_{\text{MCP}} & 0 & 0 \\ 0 & k_{\text{PIP}} & 0 \\ 0 & 0 & k_{\text{DIP}} \end{bmatrix}, \quad \mathbf{q}_{\text{eq}} \triangleq \begin{bmatrix} \varphi_{\text{MCP,eq}} \\ \varphi_{\text{PIP,eq}} \\ \varphi_{\text{DIP,eq}} \end{bmatrix}. \end{aligned} \quad (3.8)$$

The torques from the torsional dampers can be expressed as:

$$\boldsymbol{\tau}_{\text{damper}} = \begin{bmatrix} b_{\text{MCP}}\dot{\varphi}_{\text{MCP}} \\ b_{\text{PIP}}\dot{\varphi}_{\text{PIP}} \\ b_{\text{DIP}}\dot{\varphi}_{\text{DIP}} \end{bmatrix} = \mathbf{B}\dot{\mathbf{q}}, \quad \mathbf{B} \triangleq \begin{bmatrix} b_{\text{MCP}} & 0 & 0 \\ 0 & b_{\text{PIP}} & 0 \\ 0 & 0 & b_{\text{DIP}} \end{bmatrix}. \quad (3.9)$$

The total generalized forces can then be expressed as:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{ext}} + \boldsymbol{\tau}_{\text{cable}} - \boldsymbol{\tau}_{\text{spring}} - \boldsymbol{\tau}_{\text{damper}}. \quad (3.10)$$

3.1.4 Rigid-body dynamics

For rigid-body mechanical systems, like the one we're modelling, the Euler-Lagrange equations can be expressed in matrix form as (**Add reference to theory: Rigid-body dynamics**):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}. \quad (3.11)$$

The *inertia matrix* $\mathbf{M}(\mathbf{q})$ is a symmetric positive-definite matrix that represents the mass distribution of the system. We'll define $\varphi_{\text{MCP}} \triangleq \varphi_1$, $\varphi_{\text{PIP}} \triangleq \varphi_2$, and $\varphi_{\text{DIP}} \triangleq \varphi_3$, the cumulative angles $\varphi_{12} \triangleq \varphi_1 + \varphi_2$, $\varphi_{23} \triangleq \varphi_2 + \varphi_3$, and the shorthand form $c_\varphi \triangleq \cos(\varphi)$, $s_\varphi \triangleq \sin(\varphi)$. Based on the E.L.-equations the inertia matrix in our system then equates to:

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} p_1 + p_2 c_{\varphi_2} + p_3 c_{\varphi_{23}} + p_4 c_{\varphi_3} & p_4 c_{\varphi_3} + p_5 + p_6 c_{\varphi_2} + p_7 c_{\varphi_{23}} & p_7 c_{\varphi_{23}} + p_8 + p_9 c_{\varphi_3} \\ p_4 c_{\varphi_3} + p_5 + p_6 c_{\varphi_2} + p_7 c_{\varphi_{23}} & p_4 c_{\varphi_3} + p_5 & p_8 + p_9 c_{\varphi_3} \\ p_7 c_{\varphi_{23}} + p_8 + p_9 c_{\varphi_3} & p_8 + p_9 c_{\varphi_3} & p_8 \end{bmatrix}, \quad (3.12)$$

where the different p_i 's are shown in Table 3.2. The *coriolis and centrifugal matrix* $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ represents the coriolis and centrifugal forces that arise from the motion of the system. For our system, it equates to:

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2} \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} \\ \gamma_{21} & \gamma_{22} & \gamma_{23} \\ \gamma_{31} & \gamma_{32} & 0 \end{bmatrix}, \quad (3.13)$$

where

$$\begin{aligned} \gamma_{11} &= -p_2 \dot{\varphi}_2 s_{\varphi_2} - p_3 \dot{\varphi}_{2-3} s_{\varphi_{23}} - p_4 \dot{\varphi}_3 s_{\varphi_3}, \\ \gamma_{12} &= -(p_2 \dot{\varphi}_1 + 2p_6 \dot{\varphi}_2) s_{\varphi_2} - (p_3 \dot{\varphi}_1 + 2p_7 \dot{\varphi}_{2-3}) s_{\varphi_{23}} - p_4 \dot{\varphi}_3 s_{\varphi_3}, \\ \gamma_{13} &= -(p_3 \dot{\varphi}_1 + 2p_7 \dot{\varphi}_{2-3}) s_{\varphi_{23}} - (p_4 \dot{\varphi}_{1-2} + 2p_9 \dot{\varphi}_3) s_{\varphi_3}, \\ \gamma_{21} &= \dot{\varphi}_1 (p_2 s_{\varphi_2} + p_3 s_{\varphi_{23}}) - \dot{\varphi}_3 p_4 s_{\varphi_3}, \\ \gamma_{22} &= -\dot{\varphi}_3 p_4 s_{\varphi_3}, \\ \gamma_{23} &= -(p_4 \dot{\varphi}_{1-2} + 2p_9 \dot{\varphi}_3) s_{\varphi_3}, \\ \gamma_{31} &= \dot{\varphi}_1 p_3 s_{\varphi_{23}} + p_4 \dot{\varphi}_{1-2} s_{\varphi_3}, \\ \gamma_{32} &= p_4 \dot{\varphi}_{1-2} s_{\varphi_3}. \end{aligned}$$

The generalized forces $\boldsymbol{\tau}$ are the same as in Equation (3.10). The *gravity vector* $\mathbf{G}(\mathbf{q})$ represents the gravitational forces acting on the system. Since we're omitting the effects of gravity, the gravity vector is zero:

$$\mathbf{G}(\mathbf{q}) = \mathbf{0}. \quad (3.14)$$

Inserting all the results above, and solving for $\ddot{\mathbf{q}}$, we can express the equations of motion of our system as:

Table 3.2: Constants in system matrices.

Constant	Value
p_1	$J_{\text{MCP}} + J_{\text{PIP}} + J_{\text{DIP}} + \frac{l_{\text{PP}}^2}{4} m_{\text{PP}} + \left(l_{\text{PP}}^2 + \frac{l_{\text{MP}}^2}{4} \right) m_{\text{MP}} + \left(l_{\text{PP}}^2 + l_{\text{MP}}^2 + \frac{l_{\text{DP}}^2}{4} \right) m_{\text{DP}}$
p_2	$l_{\text{PP}} l_{\text{MP}} (m_{\text{MP}} + 2m_{\text{DP}})$
p_3	$l_{\text{PP}} l_{\text{DP}} m_{\text{DP}}$
p_4	$l_{\text{MP}} l_{\text{DP}} m_{\text{DP}}$
p_5	$J_{\text{PIP}} + J_{\text{DIP}} + \frac{l_{\text{MP}}^2}{4} m_{\text{MP}} + \left(l_{\text{MP}}^2 + \frac{l_{\text{DP}}^2}{4} \right) m_{\text{DP}}$
p_6	$\frac{1}{2} l_{\text{PP}} l_{\text{MP}} (m_{\text{MP}} + 2m_{\text{DP}})$
p_7	$\frac{1}{2} l_{\text{PP}} l_{\text{DP}} m_{\text{DP}}$
p_8	$J_{\text{DIP}} + \frac{l_{\text{DP}}^2}{4} m_{\text{DP}}$
p_9	$\frac{1}{2} l_{\text{MP}} l_{\text{DP}} m_{\text{DP}}$

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1} [\boldsymbol{\tau}_{\text{ext}} + \boldsymbol{\tau}_{\text{cable}} - (\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{B}) \dot{\mathbf{q}} - \mathbf{K}(\mathbf{q} - \mathbf{q}_{\text{eq}})]. \quad (3.15)$$

3.2 Brushed DC Motor model

In our system we're using a 12V Brushed DC motor from Pololu ('Pololu - Brushed DC Motor', 2026). The dynamical model is based on Rahman and Mat Yahya, 2021. The circuit diagram of a brushed DC motor is shown in Figure 3.2. Equation (3.16) describes the mechanical dynamics of the motor, and Equation (3.17) describes the electrical dynamics of the motor, defined as:

$$J_m \frac{d^2\theta}{dt^2} + B_m \frac{d\theta}{dt} = \tau_m - \tau_l \quad (3.16)$$

$$R_a I_a + L_a \frac{dI_a}{dt} + E_b = E_a. \quad (3.17)$$

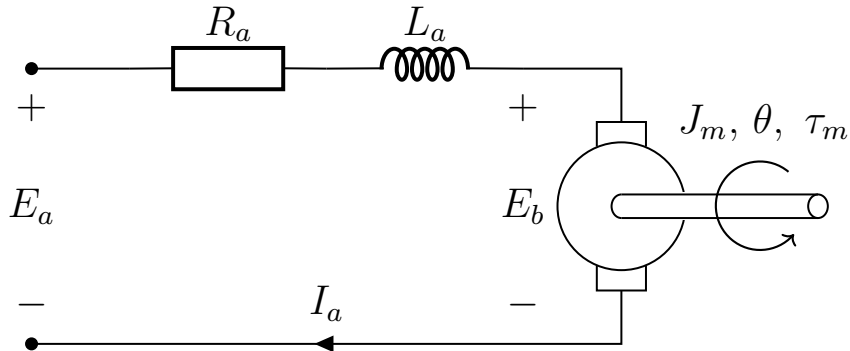


Figure 3.2: Circuit diagram of armature circuit of the DC motor model, with output torque and angular position.

Here, θ defines the angle of the motor. The torque generated by the motor is given by $\tau_m = K_t I_a$, where K_t is the torque constant of the motor. The load torque τ_l represents the torque exerted by the finger on the motor through the cable, as well as friction between the motor and the finger. The friction within the rotor of the motor is modeled with $B_m \frac{d\theta}{dt}$, where B_m is a constant friction coefficient. The inertia of the rotor is given by J_m . The back electromotive force (EMF) is given by $E_b = K_b \frac{d\theta}{dt}$, where K_b is the back EMF constant of the motor. The armature voltage E_a is the control input to the system. R_a and L_a are the armature resistance and inductance, respectively. With the change to τ_m and E_b , we can rewrite the equations as:

$$J_m \frac{d^2\theta}{dt^2} + B_m \frac{d\theta}{dt} = K_t I_a - \tau_l \quad (3.18)$$

$$R_a I_a + L_a \frac{dI_a}{dt} + K_b \frac{d\theta}{dt} = E_a. \quad (3.19)$$

3.2.1 Estimating motor parameters

To estimate the parameters of the motor, we will use the datasheet from the manufacturer ('Pololu 25D Metal Gearmotors', 2021). Relevant information extracted from the datasheet is shown in Table 3.3. Considering the motor at stall, we have $I_a = I_{a, \text{stall}}$ and $\tau_l = \tau_{l, \text{stall}}$. In addition we have $\frac{d\theta}{dt} \equiv 0$, and at stationarity, $\frac{dI_a}{dt} \equiv 0$. This allows us to rewrite Equation (3.19) as:

$$R_a I_{a, \text{stall}} = E_{a, \text{supply}} \implies R_a = \frac{E_{a, \text{supply}}}{I_{a, \text{stall}}} \approx 2.44 \, \Omega. \quad (3.20)$$

Further, when the motor is stalled, we should expect the torque generated by the motor to equal the stall torque, giving us:

$$K_t I_{a, \text{stall}} = \tau_{l, \text{stall}} \implies K_t = \frac{\tau_{l, \text{stall}}}{I_{a, \text{stall}}} \approx 0.44 \, \text{N m A}^{-1}. \quad (3.21)$$

For the case when the motor is running at no-load speed, we have $I_a = I_{a, \text{nl}}$ and $\frac{d\theta}{dt} = \omega_{\text{nl}}$, and at stationarity, $\frac{dI_a}{dt} \equiv 0$. This allows us to rewrite Equation (3.19) as:

$$R_a I_{a, \text{nl}} + K_b \omega_{\text{nl}} = E_{a, \text{supply}} \implies K_b = \frac{E_{a, \text{supply}} - R_a I_{a, \text{nl}}}{\omega_{\text{nl}}} \approx 0.845 \, \text{V s rad}^{-1}. \quad (3.22)$$

Further, at no-load speed, we have $\tau_l \equiv 0$, and $\frac{d^2\theta}{dt^2} \equiv 0$. This allows us to rewrite Equation (3.18) as:

$$B_m \omega_{\text{nl}} = K_t I_{a, \text{nl}} \implies B_m = \frac{K_t I_{a, \text{nl}}}{\omega_{\text{nl}}} \approx 6.47 \, \text{mN m s rad}^{-1}. \quad (3.23)$$

Table 3.3: Relevant motor parameters extracted from the datasheet of the motor.

Parameter	Symbol	Value
Armature Voltage	$E_{a,\text{supply}}$	12 V
Stall Torque	$\tau_{l,\text{stall}}$	2.158 N m
Stall Current	$I_{a,\text{stall}}$	4.9 A
No-load Speed	ω_{nl}	13.61 rad/sec
No-load Current	$I_{a,nl}$	0.20 A

Sadly it's not possible to estimate the inertia of the rotor J_m , nor the armature inductance L_a from the datasheet. For simulation purposes, the parameters are chosen as $J_m = 0.093 \text{ kg m}^2$ and $L_a = 6 \text{ mH}$, reported as example brushed DC motor values by Rahman and Mat Yahya (2021). Even though the values are currently unknown, they can be estimated through a calibration step later. One way that's possibly the most convenient, is to use a parameter identification method.

3.2.2 Estimating the armature inductance and rotor inertia

To estimate the armature inductance and rotor inertia, we can use a parameter identification method. We'll start by converting Equation (3.19) to the Laplace domain, giving us:

$$E_a(s) = (L_a s + R_a)I_a(s) + K_b s\theta(s). \quad (3.24)$$

Since we only measure $I_a(t)$ and not its derivative, we have to apply a proper filter of order 1, defined as:

$$\Lambda_{I_a}(s) = s + \lambda_0, \quad \lambda_0 > 0. \quad (3.25)$$

Applying the filter to Equation (3.24) gives us:

$$\frac{1}{\Lambda_{I_a}(s)}E_a(s) = \frac{1}{\Lambda_{I_a}(s)}(L_a s + R_a)I_a(s) + \frac{1}{\Lambda_{I_a}(s)}K_b s\theta(s).$$

Ordering it around, moving all measured signals and known parameters to the left, and all factors with the unknown parameter L_a to the right, we get:

$$\underbrace{\frac{1}{\Lambda_{I_a}(s)}[E_a(s)] - \frac{1}{\Lambda_{I_a}(s)}[R_a I_a(s)] - \frac{s}{\Lambda_{I_a}(s)}[K_b \theta(s)]}_{z_{L_a}} = \underbrace{\frac{s}{\Lambda_{I_a}(s)}[I_a(s)]}_{\Phi_{L_a}} \underbrace{L_a}_{\theta_{L_a}^*} \quad (3.26)$$

$$\implies z_{L_a} = \theta_{L_a}^* \Phi_{L_a}.$$

Hence we've transformed the system to a SPM like discussed in Equation (2.14). We can do the same for the inertia of the rotor J_m . We'll start by converting Equation (3.18) to the Laplace domain, assuming there is no external load ($\tau_l \equiv 0$), giving us:

$$(J_ms^2 + B_ms)\theta(s) = K_t I_a(s). \quad (3.27)$$

Since we're only measuring $\theta(t)$ and not its derivatives, we have to apply a proper filter of order 2, defined as:

$$\Lambda_\theta(s) = s^2 + \lambda_1 s + \lambda_0, \quad \lambda_0, \lambda_1 > 0. \quad (3.28)$$

Applying the filter to Equation (3.27), moving all measured signals and known parameters to the left, and all factors with the unknown parameter J_m to the right, we get:

$$\underbrace{\frac{1}{\Lambda_\theta(s)}[K_t I_a(s)] - \frac{s}{\Lambda_\theta(s)}[B_m \theta(s)]}_{z_{J_m}} = \underbrace{\frac{s^2}{\Lambda_\theta(s)}[\theta(s)]}_{\Phi_{J_m}} \underbrace{J_m}_{\theta_{J_m}^*}. \quad (3.29)$$

$$\implies z_{J_m} = \theta_{J_m}^* \Phi_{J_m}.$$

Combining Equation (3.26) and Equation (3.29), we can express them both in SPM form as:

$$\underbrace{\begin{bmatrix} z_{L_a} \\ z_{J_m} \end{bmatrix}}_{\mathbf{z}} = \underbrace{\begin{bmatrix} L_a & J_m \end{bmatrix}}_{\boldsymbol{\theta}^{*\top}} \underbrace{\begin{bmatrix} \Phi_{L_a} \\ \Phi_{J_m} \end{bmatrix}}_{\boldsymbol{\Phi}}. \quad (3.30)$$

$$\implies \mathbf{z} = \boldsymbol{\theta}^{*\top} \boldsymbol{\Phi}.$$

Using the gradient method developed in Section 2.3.5, with

$$\lambda_{0,L_a} = 10, \quad \lambda_{0,J_m} = 0.1, \quad \lambda_{1,J_m} = 0.1, \quad \boldsymbol{\Gamma} = \begin{bmatrix} 10 & 0 \\ 0 & 1000 \end{bmatrix}, \quad m_s^2 = 1,$$

$$\hat{\boldsymbol{\theta}}_0 = \begin{bmatrix} \hat{L}_a(0) \\ \hat{J}_m(0) \end{bmatrix} = \begin{bmatrix} 1 \text{ mH} \\ 0.01 \text{ kg m}^2 \end{bmatrix}, \quad \boldsymbol{\theta}^* = \begin{bmatrix} L_a \\ J_m \end{bmatrix} = \begin{bmatrix} 6 \text{ mH} \\ 0.093 \text{ kg m}^2 \end{bmatrix},$$

while applying a step input $E_a = 12 \text{ V}$ at $t = 50 \text{ ms}$, the resulting estimates of $\hat{L}_a(t)$ and $\hat{J}_m(t)$ are shown in Section 3.2.2. It is clear that the estimates converge exponentially fast to the true values. Since these are static parameters that only needs to be estimated once, L_a and J_m will be assumed known for further controller design.

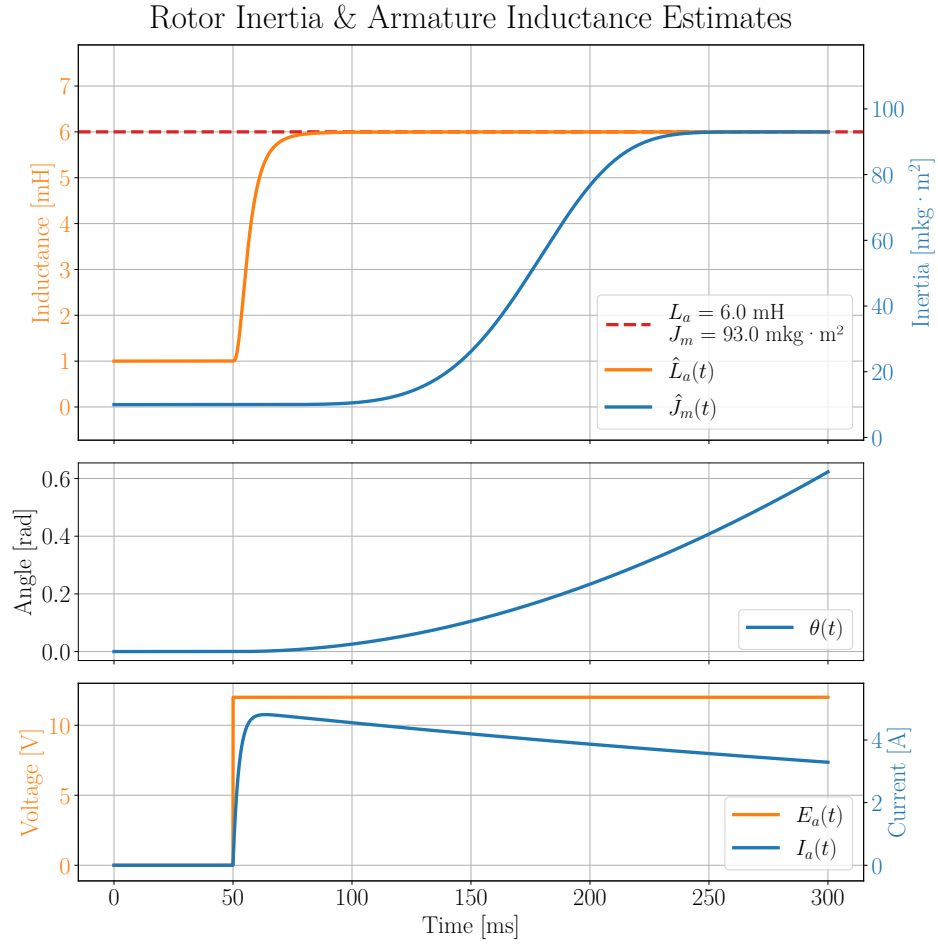


Figure 3.3: Parameter identification of the armature inductance L_a and rotor inertia J_m using a step input in the armature voltage E_a .

3.2.3 Mass-spring-damper load

In Section 3.1 we assumed that the finger could be modelled as several torsional mass-spring-damper systems. Since we can't measure the finger-angles, in addition to that the configuration of the whiffle tree is unknown, the full finger model isn't observable, and can't be used for position control design. Instead, we'll assume that finger can be modelled as a single mass-spring-damper system, acting on the motor as τ_l . This is a simplification, but it allows for a better analysis of the system. Hence we'll model the load torque τ_l as:

$$\tau_l = J \frac{d^2\theta}{dt^2} + c \frac{d\theta}{dt} + k(\theta - \theta_{eq}), \quad (3.31)$$

where J is the 'total' inertia of the finger, c is the 'total' damping coefficient of the finger, k is the 'total' stiffness of the finger, and θ_{eq} is a kind of equilibrium angle of the finger. Substituting this into Equation (3.18), we get the updated mechanical dynamics:

$$(J_m + J) \frac{d^2\theta}{dt^2} + (B_m + c) \frac{d\theta}{dt} + k\theta = K_t I_a + k\theta_{eq} \quad (3.32)$$

3.2.4 State-space model of brushed DC motor with load

To analyze the position controller for the motor with load, discussed further in Section 4.3, we need to express the dynamics in Equation (3.32) in state-space representation, as shown in Section 2.3.2. Assuming that I_a can be treated as an input, explained in Section 4.1, and defining $\omega \triangleq \frac{d\theta}{dt}$, we can rewrite the Equation (3.32) and as:

$$\frac{d\omega}{dt} = \frac{1}{J_m + J} (K_t I_a - (B_m + c)\omega - k(\theta - \theta_{eq})). \quad (3.33)$$

Written in state-space form, we have:

$$\underbrace{\frac{d}{dt} \begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\frac{d}{dt} \mathbf{x}} = \underbrace{\begin{bmatrix} 0 & 1 \\ -\frac{k}{J_m+J} & -\frac{B_m+c}{J_m+J} \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\mathbf{x}} + \underbrace{\begin{bmatrix} 0 \\ \frac{K_t}{J_m+J} \end{bmatrix}}_{\mathbf{B}} \underbrace{I_a}_{\mathbf{u}} + \underbrace{\begin{bmatrix} 0 \\ \frac{k\theta_{eq}}{J_m+J} \end{bmatrix}}_{\mathbf{d}} \quad (3.34)$$

$$\implies \frac{d}{dt} \mathbf{x} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} + \mathbf{d}.$$

In addition we have an encoder measuring θ . Hence the output of the system is given by:

$$\underbrace{\begin{bmatrix} \theta \end{bmatrix}}_{\mathbf{y}} = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{\mathbf{C}} \underbrace{\begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\mathbf{x}} \quad (3.35)$$

$$\implies \mathbf{y} = \mathbf{C}\mathbf{x}.$$

Chapter 4

Controller design

4.1 Current controller

To help facilitate the design of the position and torque controllers, we will first design a controller for the armature current I_a with the armature voltage E_a . This will allow us to treat the armature current as a control input in the position control design, simplifying the dynamics significantly. As the dynamics of the armature current is so fast, the controller has to be as cheap as possible to compute. Hence, it will be designed as a simple proportional (P) controller, but with a feedforward (FF) term to account for the back EMF. Let's first consider the electrical dynamics of the motor from Equation (3.19):

$$L_a \frac{dI_a}{dt} + R_a I_a + K_b \frac{d\theta}{dt} = E_a. \quad (4.1)$$

Given the time varying reference current $r(t)$, we can design the input voltage $E_a(t)$ as:

$$E_a(t) = \underbrace{K_b \frac{d\theta(t)}{dt}}_{\text{Feedforward term}} + \underbrace{L_a K_p (r(t) - I_a(t))}_{\text{P controller}}. \quad (4.2)$$

Substituting Equation (4.2) into Equation (4.1), we get:

$$\frac{dI_a(t)}{dt} + R_a I_a(t) = K_p (r(t) - I_a(t)) \triangleq u(t). \quad (4.3)$$

This controller is supposed to be ran in real time on a microcontroller. That way it won't have access to continuous time signals, but rather discrete time signals. Hence, we will need to discretize the controller for stability analysis and tuning. For this analysis we will assume that the dynamics of the motor are slow enough such that the feedforward term can be treated as a constant. This is a reasonable assumption since the feedforward term is based on the angular velocity, which changes relatively slowly

compared to the armature current.

4.1.1 Discretization of current controller

Suppose the controller is ran with a update period of T_s . Assuming that the input voltage E_a is held constant between sampling times, i.e.,

$$u(t) = u[k] = K_p(r[k] - I_a[k]), \quad kT_s \leq t < (k+1)T_s, \quad (4.4)$$

the dynamics in Equation (4.3) then becomes:

$$\frac{dI_a(\tau)}{d\tau} + R_a I_a(\tau) = u[k], \quad \tau = t - kT_s \in [0, T_s]. \quad (4.5)$$

To solve it, let's multiply by the integrating factor $e^{R_a\tau}$, which gives:

$$e^{R_a\tau} \left[\frac{dI_a(\tau)}{d\tau} + R_a I_a(\tau) \right] = u[k] e^{R_a\tau}. \quad (4.6)$$

Notice how this can be rewritten as:

$$\frac{d}{d\tau} \left(e^{R_a\tau} I_a(\tau) \right) = u[k] e^{R_a\tau}. \quad (4.7)$$

Integrating both sides of Equation (4.7) with respect to τ , over the entire interval from 0 to T_s , we get the discretized dynamics of the current controller:

$$\begin{aligned} [e^{R_a\tau} I_a(\tau)]_{\tau=0}^{\tau=T_s} &= u[k] \int_0^{T_s} e^{R_a\tau} d\tau, \\ \implies e^{R_aT_s} \underbrace{I_a(T_s)}_{I_a[k+1]} - \underbrace{I_a(0)}_{I_a[k]} &= u[k] \frac{e^{R_aT_s} - 1}{R_a}, \\ \implies I_a[k+1] &= \underbrace{e^{-R_aT_s}}_{\triangleq a} I_a[k] + u[k] \underbrace{\frac{1 - e^{-R_aT_s}}{R_a}}_{\triangleq b}, \\ \implies I_a[k+1] &= aI_a[k] + bu[k]. \end{aligned} \quad (4.8)$$

Substituting in $u[k]$ from Equation (4.4), we get:

$$\begin{aligned} I_a[k+1] &= aI_a[k] + bK_p(r[k] - I_a[k]), \\ \implies I_a[k+1] &= (a - bK_p) I_a[k] + bK_p r[k]. \end{aligned} \quad (4.9)$$

4.1.2 Stability analysis and tuning of controller

To guarantee stability of the current controller, we can look at the z-transform of the system. Taking the z-transform of Equation (4.9), disregarding any initial conditions, we get:

$$\begin{aligned}
[z - (a - bK_p)] I_a(z) &= bK_p r(z), \\
\implies \frac{I_a(z)}{r(z)} &= \frac{bK_p}{z - (a - bK_p)}.
\end{aligned} \tag{4.10}$$

Clearly, Equation (4.10) has a pole at $z = a - bK_p$. As discussed in Section 2.3.1, for the system to be stable, we need the pole to be inside the unit circle. In other words, we need:

$$\begin{aligned}
|a - bK_p| < 1 &\implies \left| e^{-R_a T_s} - K_p \frac{1 - e^{-R_a T_s}}{R_a} \right| < 1, \\
\implies K_p &< R_a \frac{1 + e^{-R_a T_s}}{1 - e^{-R_a T_s}} \triangleq K_{p,\max}(T_s).
\end{aligned} \tag{4.11}$$

One downside to using a P controller is that there will be a steady-state error. To eliminate this, we can artificially give the controller a higher reference current. To do this we will need to know the steady-state value of the current for a given reference. Assuming we apply a step reference of $r[k] = r_0$ for all $k \geq 0$. Provided we choose a K_p that satisfies Equation (4.11), the recursion will converge, and as we let $t \rightarrow \infty \implies k \rightarrow \infty$, Equation (4.9) becomes:

$$\begin{aligned}
I_a[\infty] &= (a - bK_p)I_a[\infty] + bK_p r_0, \\
\implies I_a[\infty] &= r_0 \frac{bK_p}{1 - a + bK_p} = r_0 \frac{\frac{1-a}{R_a} K_p}{1 - a + \frac{1-a}{R_a} K_p} = r_0 \frac{K_p}{R_a + K_p}.
\end{aligned} \tag{4.12}$$

Hence to eliminate the steady-state error, we can choose the reference to be:

$$r_1 = \frac{R_a + K_p}{K_p} r_0 \implies I_a[\infty] = r_1 \frac{K_p}{R_a + K_p} = r_0. \tag{4.13}$$

4.1.3 Simulation of current controller

To evaluate the controller, we will simulate it applied to the motor model developed in Section 3.2. Using $K_p = \frac{1}{2}K_{p,\max}(T_s)$, where the reference current is a step input of 1 A, and scaled with regards to Equation (4.13). Additionally there is a saturation put on $E_a = \pm 12$ V. Figure 4.1 and Figure 4.2 shows a simulation of the P+FF controller aswell as just the P controller, with $T_s = 0.2$ ms, where the motor is initially spinning at $\omega_0 = 5 \text{ rad s}^{-1}$ and $\omega_0 = 0 \text{ rad s}^{-1}$, respectively. Figure 4.3 and Figure 4.4 shows the simulation of the P+FF controller with and without noise to the measurement of I_a , with $T_s = 0.2$ ms and $T_s = 1.0$ ms, respectively. The noise is a zero-mean Gaussian noise with a standard deviation of $\sigma_{I_a} = 50$ mA, which is higher than is expected for the system, but it will help us evaluate the robustness of the controller.

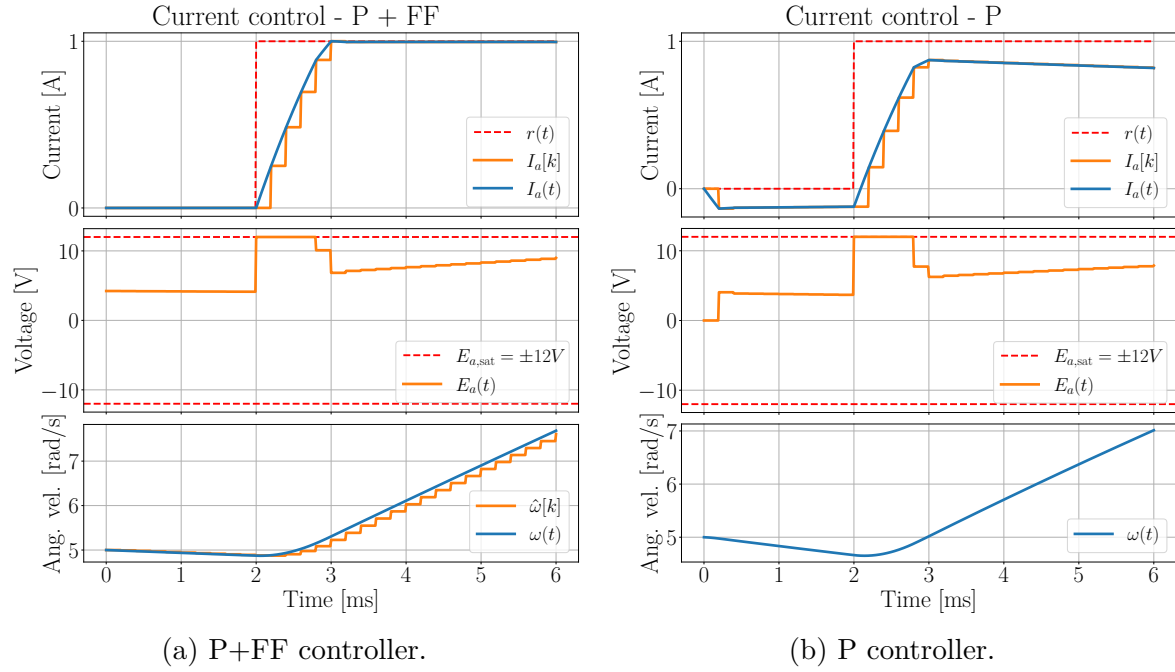


Figure 4.1: Armature current step response for discrete-time P+FF and P controllers tracking a 1 A reference, with $T_s = 0.2$ ms and initial motor speed $\omega_0 = 5$ rad s⁻¹.

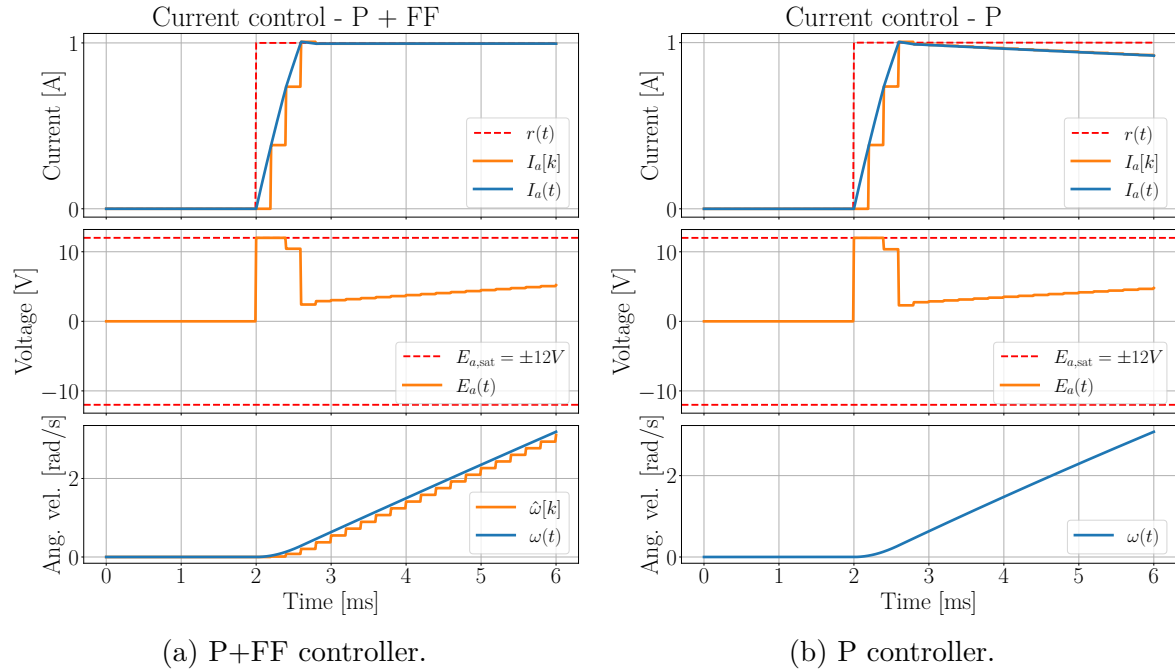


Figure 4.2: Armature current step response for discrete-time P+FF and P controllers tracking a 1 A reference, with $T_s = 0.2$ ms and initial motor speed $\omega_0 = 0$ rad s⁻¹.

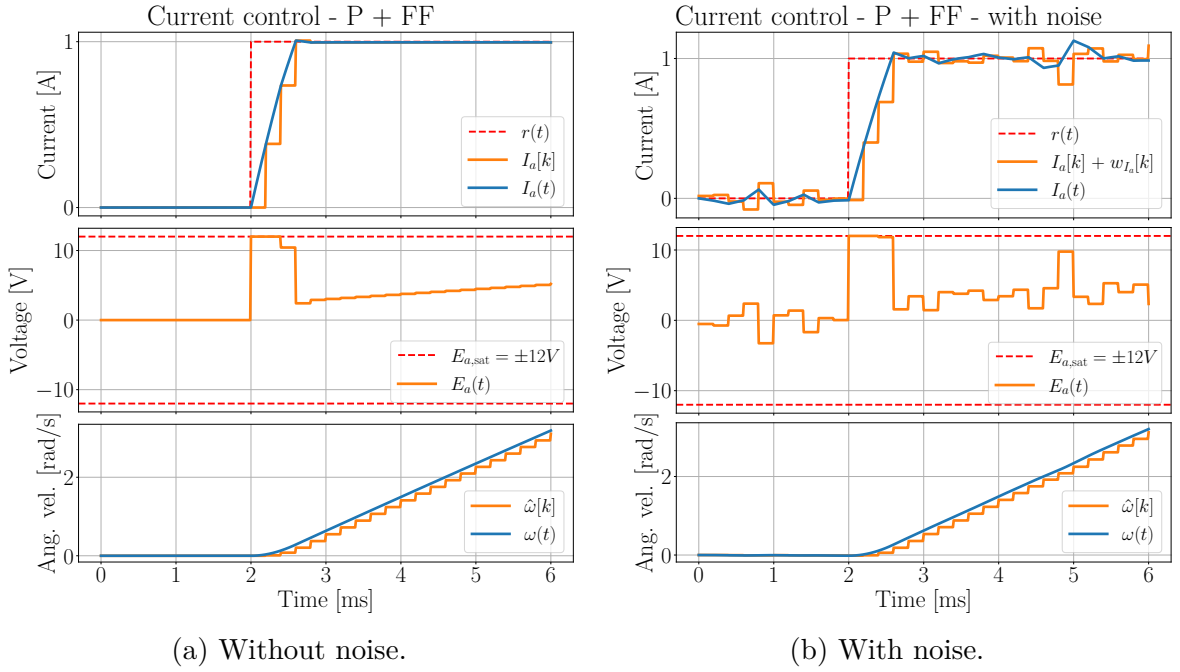


Figure 4.3: Effect of current measurement noise on the armature current response using a discrete-time P+FF controller with $T_s = 0.2$ ms.

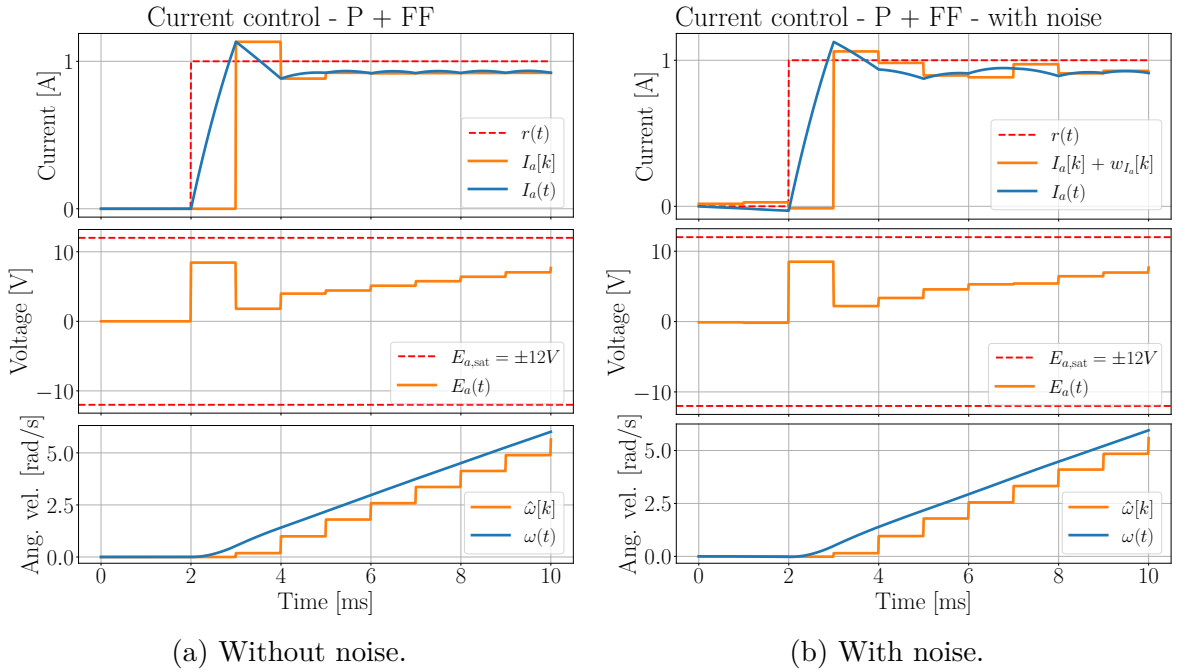


Figure 4.4: Effect of current measurement noise on the armature current response using a discrete-time P+FF controller with $T_s = 1.0$ ms.

4.1.4 Evaluation of controller

When the motor is starting from still, both the P+FF and P controllers are able to reach the target current fast. But as the motor picks up speed, the back-EMF becomes significant, and the P controller struggles to keep up, and the current starts to deviate from the reference current, as can be seen in Figure 4.2b. The P+FF controller on the other hand continues to perform well, as it can account for the back-EMF, as shown in Figure 4.2a. When the motor is already spinning at a high speeds, the differences between the two controllers becomes even more significant. At no point does the P controller reach its target current. In fact, it deviates more and more over time, as can be seen in Figure 4.1b. The P+FF controller on the other hand continues to perform well, as it can account for the back-EMF, as shown in Figure 4.1a.

As can be seen in Figure 4.3 and Figure 4.4, the added noise to the current measurements doesn't really affect the stability of the controller. The current oscillates slightly when there's noise present, compared to the noise-free. There is however a clear difference between longer and shorter update periods. With a longer update period of $T_s = 1.0$ ms, the current gets a slight steady-state-error, which is not present with the shorter update period of $T_s = 0.2$ ms. The reason for this is that the assumption of a constant speed between updates is less valid with a longer update period, which causes the feedforward term to be less accurate, which in turn causes the steady-state error. With a shorter update period, the assumption of a constant speed is more valid, which allows the feedforward term to be more accurate, which in turn allows the controller to track the reference current better.

Overall, the P+FF controller shows promising results for controlling the armature current in our system. Due to the saturation in the voltage of $E_a = \pm 12$ V, there is a maximum rate of change for the current. During simulation it was found to be $\dot{I}_{a_{\max}} \approx 2.0 \text{ kA s}^{-1}$, when the motor was starting from still, which is more or less instantaneous seen from the dynamics of the mechanical system. Hence, we will treat the armature current as a control input for further control design.

4.2 Torque control

When the hand interacts with objects, trying to control the position of the hand becomes irrelevant, as the hand will be constrained by the object. Instead, we want to control the torque generated by the motor, which in turn will allow us to control the force applied to the object.

When grasping an object, assuming the object is rigid and that the cable is inelastic, the angular velocity and acceleration of the motor will be $\frac{d\theta}{dt} \equiv 0$ and $\frac{d^2\theta}{dt^2} \equiv 0$, respectively. Based on Equation (3.18), this results in the following relation:

$$\tau_m = K_t I_a = \tau_l. \quad (4.14)$$

Hence, all the torque generated by the motor, which is proportional to the current, would be applied to the load. This means the applied torque can be controlled directly through the current. In Section 4.1 we designed a controller for the armature current I_a through the input armature voltage E_a . With $\frac{d\theta}{dt} \equiv 0$, the back-EMF will be $K_b \frac{d\theta}{dt} \equiv 0$. Therefore we can also simplify the current controller in Equation (4.2) to just a P controller, as there is no back-EMF to account for. This will reduce the computational load on the microcontroller. For a given reference torque r_{τ_m} , we can calculate the reference current r_{I_a} as:

$$r_{I_a} = \frac{r_{\tau_m}}{K_t}. \quad (4.15)$$

If it's more convenient to control the force applied to the object instead of the torque, we can easily convert between the two using the relation $\tau_m = F_m r_{\text{spindle}}$, where r_{spindle} is the radius of the spindle mounted on the motor. Hence, for a given reference force r_F , we can calculate the reference current r_{I_a} as:

$$r_{I_a} = r_F \frac{r_{\text{spindle}}}{K_t}. \quad (4.16)$$

Figure 4.5 shows emulated scenarios of the finger grasping a rigid object, assuming no elasticity in the cable, and no friction between the motor and the finger. The motor is supplied with 3 V for the first 5 ms. Then at $t = 5$ ms, the finger makes contact with the object, and $\frac{d\theta}{dt} = \frac{d^2\theta}{dt^2} \equiv 0$. At the same time, the torque controller is activated, and the reference torque is set to $r_{\tau_m} = 1.5$ N m. For the current controller, the proportional gain is set to $K_p = \frac{1}{2}K_{p,max}$, like in Section 4.1.3. Figure 4.5a and Figure 4.5b show the results for $T_s = 0.2$ ms and $T_s = 1.0$ ms, respectively. As can be seen in the figures, the motor torque τ_m quickly converges to the reference torque r_{τ_m} for both cases.

Ideally, if there was no friction between the motor and the finger, $\tau_l = \tau_{\text{finger}}$. In reality, because of the bowden tube the cable is running through, and other inconsistencies in the setup, there will be some friction. Therefore, we can expect $\tau_l = \tau_{\text{finger}} + \tau_{\text{friction}}$, where τ_{friction} is the torque lost to friction. Hence, without any direct measurement of the applied force to the finger, we would expect that the applied force to the finger will be less than the reference force. However, with integrated force sensors in the fingertip, this wouldn't be an issue.

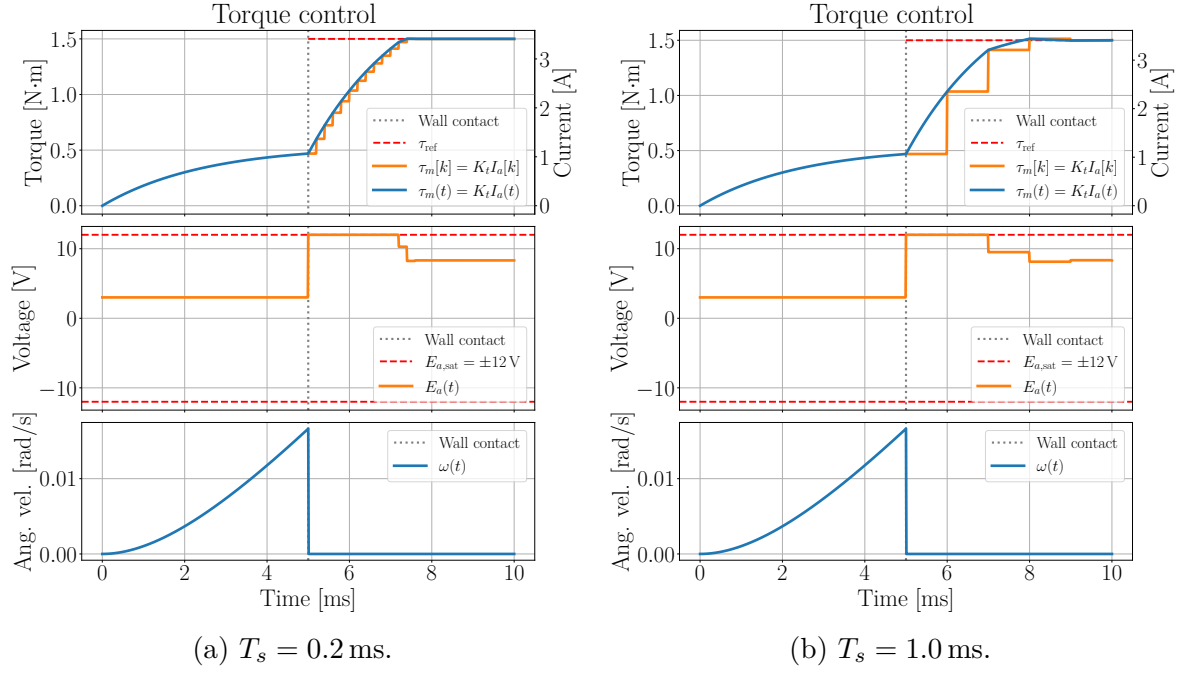


Figure 4.5: Torque control with a P controller when gripping a rigid object.

4.2.1 Force control with integrated fingertip force sensors

With integrated force sensors in the fingertip, we could use a cascade control approach. Since the motor torque and force are proportional to the armature current, we could then define the reference current r_{I_a} as a feedforward and PI controller with the reference force r_F and the measured force F_{meas} as input:

$$r_{I_a}(t) = \underbrace{\frac{r_F r_{\text{spindle}}}{K_t}}_{\text{Feedforward}} + \underbrace{K_p(r_F - F_{meas}(t)) + K_i \int_{t_0}^t (r_F - F_{meas}(\tau)) d\tau}_{\text{PI controller}}. \quad (4.17)$$

In our system, the cables are running through bowden tubes. This introduces a significant amount of friction. Therefore the original assumption in Equation (4.14) that all the torque generated by the motor is applied to the load, becomes invalid. Jeong and Cho, 2015 proposes the friction from cables going through bowden tubes to be proportional with the applied force. We then get the following relation:

$$F_{\text{finger}} = F_m (1 - \exp(-\mu\Phi)), \quad (4.18)$$

where μ is the friction coefficient, and Φ is the total wrap angle of the cable around the bowden tube. In theory one could include this in the feedforward term, but for the time being, I'll assume it to be unknown. For simulation purposes, I'll assume that $\exp(-\mu\Phi) \equiv 0.1$. This means that only 90% of the motor force is applied to the finger. Figure 4.6 shows the simulation of the force control with and without the integrated

fingertip force sensors, with $T_s = 0.2$ ms and $r_F = 100$ N. Here the controller parameters were chosen simply through repeated simulation, where $K_p = 0.001$ and $K_i = 8$. These will most likely have to be retuned in the actual system. As can be seen in Figure 4.6a, without the integrated fingertip force sensors, the applied force to the finger never reaches its target, as is expected due to the friction. With the integrated fingertip force sensors, as shown in Figure 4.6b, the applied force to the finger quickly converges to the reference force, despite the friction.

In the simulation, the resultant force after the friction was assumed directly applied to the fingertip. In the real system however, a whiffle tree will be used. It might therefore be useful to scale the feedforward term to account for the total force the motor will have to pull on the whiffle tree, such that the reference force is applied to the fingertip. Given that the force ratio for the distal phalanx is w_3 , the feedforward term could instead be defined as:

$$r_{Ia}(t) = \underbrace{\frac{1}{w_3} \frac{r_F r_{\text{spindle}}}{K_t}}_{\text{Scaled feedforward}} + \underbrace{K_p(r_F - F_{meas}(t)) + K_i \int_{t_0}^t (r_F - F_{meas}(\tau)) d\tau}_{\text{PI controller}}. \quad (4.19)$$

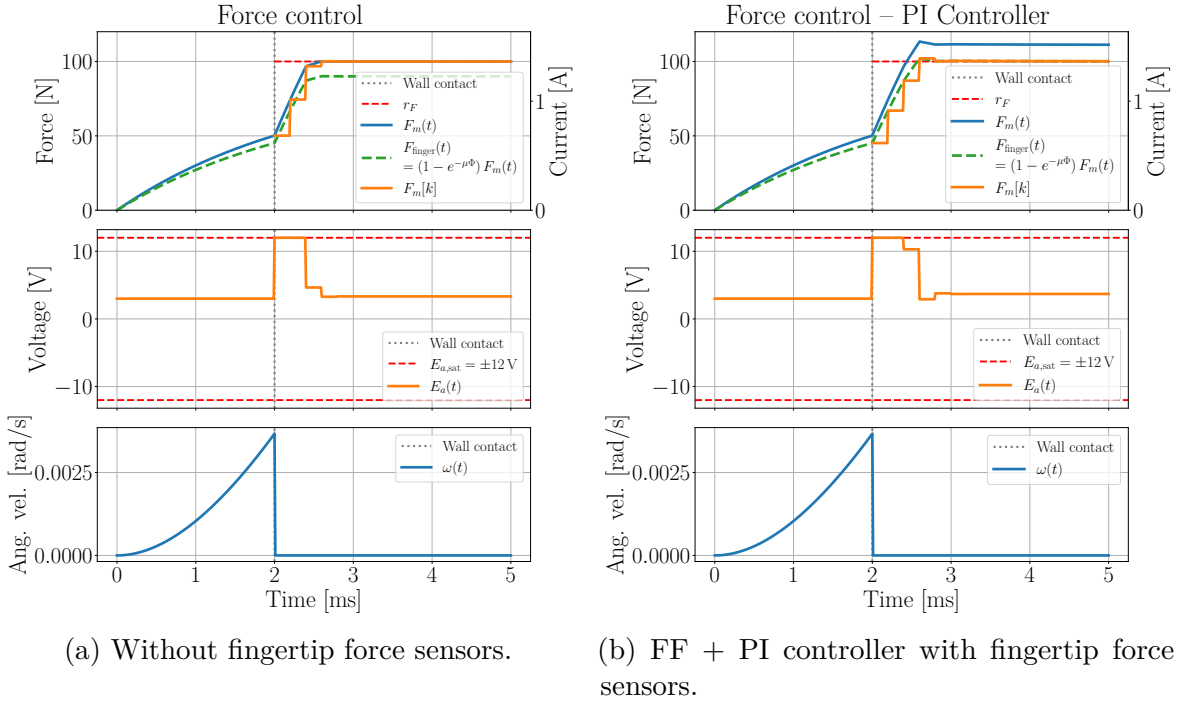


Figure 4.6: Force control with friction. With and without force sensors in fingertips. Both controllers have $T_s = 0.2$ ms.

4.3 Model reference adaptive control (MRAC) for position control

When the hand is not in contact with any objects, we want to control the position of the hand. However, due to the big differences in mass, stiffness and dampening in the hand between patients, it becomes difficult to design a controller that can guarantee good performance for all patients. To address this issue, we will implement a model reference adaptive control (MRAC) scheme, which will allow us to adapt the controller parameters online based on the observed behavior of the system. To be clear, this is mostly just to speed up the tuning process of the controller, and get good enough control for tests when frequently switching between patients. I would however recommend to use a simpler control approach if the system is to be used several times by the same patient. This will be discussed in more detail in Chapter 5.

As discussed in Section 3.2.3, because of modelling uncertainties and the fact that we don't measure anything at the hand, the finger model from Section 3.1 becomes unobservable. Therefore controlling the angles of the fingers directly becomes infeasible. Instead we will focus on controlling the position of the motor precisely, as it's connected directly to the finger through a cable. For this we will use the MRAC scheme introduced in Section 2.3.6. It calls for the system to be controllable. Using the simplified model of the finger dynamics as a torsional mass-spring-damper system acting on the motor, from Section 3.2.4, we can check if the simplified system is controllable.

4.3.1 Controllability of the system

Based on the state-space representation our system in Section 3.2.4, we can check if the system is controllable, as described in Section 2.3.3. The controllability matrix $\mathcal{C}$ for our system is given by:

$$\mathcal{C} = [\mathbf{B}, \mathbf{AB}]. \quad (4.20)$$

Substituting for our specific $\mathbf{A}$ and $\mathbf{B}$, we get:

$$\mathcal{C} = \begin{bmatrix} 0 & \frac{K_t}{J_m + J} \\ \frac{K_t}{J_m + J} & -\frac{K_t(B_m + c)}{(J_m + J)^2} \end{bmatrix}. \quad (4.21)$$

Clearly, $\text{rank}(\mathcal{C}) = 2$, which is equal to the number of states. Therefore, the simplified system is controllable.

4.3.2 MRAC implementation

The MRAC scheme that will be implemented is based on Section 2.3.6. In our physical model of the system, we will use the finger model developed in Section 3.1, connected to the brushed DC motor as a load, through a cable connected to a whiffle tree to spread the force to all three phalanges. From the motor side, the dynamics can be described through the state-space representation:

$$\underbrace{\frac{d}{dt} \begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\dot{\mathbf{x}}} = \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & -\frac{B_m}{J_m} \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\mathbf{x}} + \underbrace{\begin{bmatrix} 0 \\ \frac{K_t}{J_m} \end{bmatrix}}_{\mathbf{B}} \underbrace{I_a}_u + \underbrace{\begin{bmatrix} 0 \\ \frac{\tau_l}{J_m} \end{bmatrix}}_{\mathbf{d}}, \quad y(t) = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{\mathbf{C}} \underbrace{\begin{bmatrix} \theta \\ \omega \end{bmatrix}}_{\mathbf{x}}, \quad (4.22)$$

where θ is the angle of the motor, ω is the angular velocity of the motor, I_a is the armature current, and τ_l is the load torque applied to the motor by the finger. The output $y(t)$ is the angle of the motor, which we want to control. The MRAC scheme calls for a reference model, which the system should follow as closely as possible. Since our system is a second-order system, we will use a second-order reference model, in the standard form:

$$\underbrace{\frac{d}{dt} \begin{bmatrix} \theta_m \\ \omega_m \end{bmatrix}}_{\dot{\mathbf{x}}_m} = \underbrace{\begin{bmatrix} 0 & 1 \\ -\omega_n^2 & -2\zeta\omega_n \end{bmatrix}}_{\mathbf{A}_m} \underbrace{\begin{bmatrix} \theta_m \\ \omega_m \end{bmatrix}}_{\mathbf{x}_m} + \underbrace{\begin{bmatrix} 0 \\ \omega_n^2 \end{bmatrix}}_{\mathbf{B}_m} r_\theta, \quad (4.23)$$

where ω_n is the natural frequency of the system, ζ is the damping ratio, and r_θ is the reference angle for the motor that we want the system to follow. The MRAC scheme requires that the reference model is stable. The eigenvalues of $\mathbf{A}_m$ in Equation (4.23) are given by:

$$\lambda = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}. \quad (4.24)$$

Therefore, as long as $\zeta, \omega_n > 0$, the reference model will be stable. Similarly to the torque controller, we'll use the current controller designed in Section 4.1, such that we can treat I_a as the control input for the MRAC scheme. We can therefore define the reference current r_{I_a} as:

$$r_{I_a}(t) = -\mathbf{K}^\top(t)\mathbf{x}(t) + L(t)r_\theta(t), \quad (4.25)$$

where $\mathbf{K}(t) \in \mathbb{R}^2$ and $L(t) \in \mathbb{R}$ are the adaptive parameters that will be updated online based on the observed behavior of the system. Defining the tracking error $\mathbf{e}(t)$ as:

$$\mathbf{e}(t) = \mathbf{x}(t) - \mathbf{x}_m(t). \quad (4.26)$$

Solving for $\mathbf{P}$ for a $\mathbf{Q} = \mathbf{Q}^\top > 0$ in the Lyapunov equation:

$$\mathbf{A}_m^\top \mathbf{P} + \mathbf{P} \mathbf{A}_m = -\mathbf{Q}, \quad (4.27)$$

we can define the adaptive law as:

$$\begin{aligned} \dot{\mathbf{K}}(t) &= \mathbf{\Gamma}_K \mathbf{x} \mathbf{B}_m^\top \mathbf{P} \mathbf{e}, \\ \dot{L}(t) &= -\gamma_L \mathbf{B}_m^\top \mathbf{P} \mathbf{e} r_\theta(t), \end{aligned} \quad (4.28)$$

where $\mathbf{\Gamma}_K \in \mathbb{R}^{2 \times 2}$ and $\gamma_L \in \mathbb{R}$ are positive definite matrices that determine the adaptation rate of the parameters.

4.3.3 Connecting the motors to the finger through a cable

For the simulation of the position controller, we will simulate two motors, one for flexion and one for extension, as is the case in our physical system. The motors will be pulling on each their cables, where force the motors generate gets distributed to the three phalanges through their own whiffle tree, with $w_1 = \frac{1}{2}$ of the force going to the proximal phalanx, $w_2 = \frac{1}{3}$ of the force going to the middle phalanx, and $w_3 = \frac{1}{6}$ of the force going to the distal phalanx. The force application is shown in Figure 4.7.

Cable tension

To emulate the cable connecting the motor to the finger, the following constraint is added to the system:

$$g(\theta, \mathbf{q}) \triangleq r_{spindle} \theta + L_{finger}(\mathbf{q}) - C = 0, \quad \mathbf{q} = \begin{bmatrix} \varphi_{MCP} \\ \varphi_{PIP} \\ \varphi_{DIP} \end{bmatrix}, \quad (4.29)$$

where $r_{spindle}$ is the radius of the spindle mounted on the motor, θ is the angle of the motor, $L_{finger}(\mathbf{q})$ is an estimate for the total length of cable from the finger to the whiffle tree. C is a constant defining the total cable length in the system, which then constraints the angle of the motor and the relative angles of the finger. The estimate of the total cable going from the finger to the whiffle tree is defined as:

$$L_{finger}(\mathbf{q}) = \sum_{i=1}^3 w_i \|\mathbf{r}_{a,i} - \mathbf{r}_{t,i}\|, \quad (4.30)$$

where $\mathbf{r}_{a,i}$ and $\mathbf{r}_{t,i}$ are the attachment and target points of the i -th phalanx, respectively, shown in Figure 4.7. The distance between the attachment and target points of each phalanx is scaled with the force ratios w_i to get a better estimate of the total length of cable with the whiffle tree. Differentiating the constraint, we get the velocity constraint:

$$\dot{g}(\omega, \mathbf{q}, \dot{\mathbf{q}}) = r_{spindle}\omega + \frac{\partial L_{finger}}{\partial \mathbf{q}} \dot{\mathbf{q}} = 0, \quad (4.31)$$

and further once more we get the acceleration constraint:

$$\ddot{g}(\dot{\omega}, \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = r_{spindle}\dot{\omega} + \frac{\partial L_{finger}}{\partial \mathbf{q}} \ddot{\mathbf{q}} + \dot{\mathbf{q}}^\top \frac{\partial^2 L_{finger}}{\partial \mathbf{q}^2} \dot{\mathbf{q}} = 0. \quad (4.32)$$

To reduce numerical drift, Baumgarte stabilization is added, to combine all the constraints into a single equation:

$$\ddot{g} + 2\alpha\dot{g} + \beta^2 g = 0, \quad \alpha, \beta > 0. \quad (4.33)$$

The final constraint in Equation (4.33) is defined separately for the flexion and extension cables. The corresponding Lagrange multipliers are treated as the cable tensions T_f and T_e for the flexion and extension cable, respectively, and are solved simultaneously at each timestep.

Since the cables only can pull on the finger, we further impose that the cable tensions must be non-negative:

$$T_f, T_e \geq 0. \quad (4.34)$$

To be clear, this implementation of the cable tension is not a perfect emulation of the real system. In reality, the cable is slightly elastic, and the whiffle trees will provide some friction. There will also be more cable in the system. However, considering that the position of the attachment points, target points, and configuration of whiffle trees are not yet decided in the real system, this should provide a good enough approximation to test the position controller, and get a better understanding of the dynamics of the system.

Updated dynamics with the cable tension

Given the cable tension T_x prior to the whiffle tree for the x -th cable, the x -th brushed DC motor experiences a load torque of $\tau_l = r_{spindle}T_x$, giving the updated mechanical dynamics from Equation (3.18):

$$J_m \frac{d^2\theta_x}{dt^2} + B_m \frac{d\theta_x}{dt} = K_t I_a - r_{spindle}T_x. \quad (4.35)$$

Similarly, the finger experiences a generalized load torque. Since the cable tension acts along the line from the attachment point $\mathbf{r}_{a,i}$ to the target point $\mathbf{r}_{t,i}$, the force applied to the i -th phalanx is

$$\mathbf{F}_{x,i} = w_i T_x \frac{\mathbf{r}_{x,t,i} - \mathbf{r}_{x,a,i}}{\|\mathbf{r}_{x,t,i} - \mathbf{r}_{x,a,i}\|}. \quad (4.36)$$

The generalized torque contribution from this force is computed with:

$$\boldsymbol{\tau}_{x,i} = \frac{\partial(\mathbf{r}_{x,a,i} - \mathbf{r}_{x,t,i})^\top}{\partial \mathbf{q}} \mathbf{F}_{x,i}, \quad (4.37)$$

which leads to the total generalized cable torque on the finger:

$$\boldsymbol{\tau}_{x,cable} = \sum_{i=1}^3 \boldsymbol{\tau}_{x,i}. \quad (4.38)$$

Given that we have two motors pulling on the finger with their own cables, we can calculate the total generalized torque from both the flexion and extension cables on the finger as:

$$\boldsymbol{\tau}_{cable} = \sum_{i=1}^3 \frac{\partial(\mathbf{r}_{f,a,i} - \mathbf{r}_{f,t,i})^\top}{\partial \mathbf{q}} \mathbf{F}_{f,i} + \sum_{i=1}^3 \frac{\partial(\mathbf{r}_{e,a,i} - \mathbf{r}_{e,t,i})^\top}{\partial \mathbf{q}} \mathbf{F}_{e,i}. \quad (4.39)$$

This gives us the updated finger dynamics from Equation (3.15):

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + (\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{B})\dot{\mathbf{q}} + \mathbf{K}(\mathbf{q} - \mathbf{q}_{eq}) = \boldsymbol{\tau}_{ext} + \boldsymbol{\tau}_{cable}. \quad (4.40)$$

4.3.4 Simulation of MRAC scheme with updated dynamics

Equation (4.40) is used for simulating the dynamics of the finger, while two sets of Equation (4.35) are used for simulating the dynamics of the two motors. To connect the models, a cable constraint is added to emulate the cables connecting the motors and the finger, as discussed in Section 4.3.3. The parameters for the motor and finger model are given in Table 4.1. The estimation of the motor parameters are done in Section 3.2.1. The parameters of the finger model are not taken from any literature, but were selected empirically through repeated simulations of the unloaded finger to obtain realistic motion.

Both cables are initialized to be under tension when the finger is in a straight position, i.e. when $\varphi_{MCP} = \varphi_{PIP} = \varphi_{DIP} = 0$. The flexion motor is initialized to $\theta_f(0) = 0$ rad, while the extension motor is initialized to $\theta_e(0) = \frac{\pi}{2}$ rad. To make sure the motors aren't working against each other, they receive an opposite reference angle. This ensures that as one motor pulls on the finger, increasing the tension in its cable, the other motor will be rotating in the opposite direction, decreasing the tension in its cable. When the motors are decreasing the tension in their cables, the motor would experience $\tau_l \approx 0$ N m. Therefore the dynamics would change, throwing off the adaptive gains. To account for this, we switch to a pretuned (PT) controller, tuned for a unloaded motor. At the same time we turn off the adaptive law. Once the motor receives a reference that will increase the tension in the cable, we switch back to the adaptive controller, and turn on the adaptive law. The pretuned controller is tuned through the exact

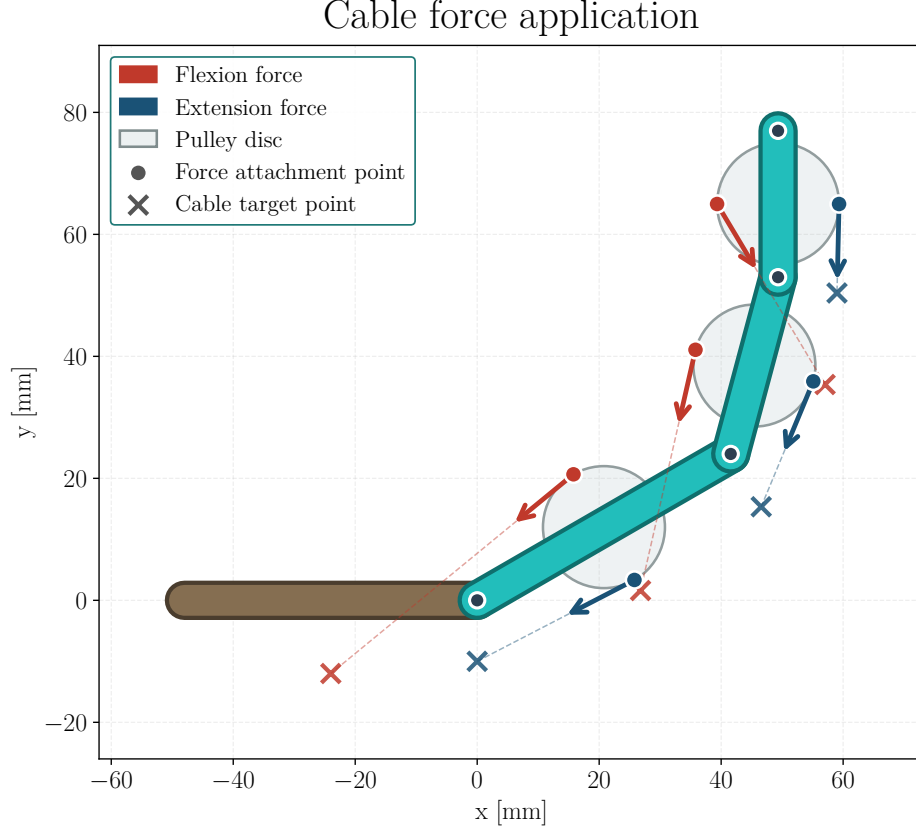


Figure 4.7: Force application on the finger. The flexion motor applies a force F_f on the flexion cable, which gets distributed to the three phalanges through the whiffle tree. Similarly, the extension motor applies a force F_e on the extension cable, which also gets distributed to the three phalanges through its own whiffle tree.

same adaptive controller, just with an unloaded motor for 100s. When the pretuned controller is active, these final values of $\mathbf{K}$ and L are used instead of the adaptive gains. The reference angles for the flexion motor and the extension motor is given by:

$$r_{\theta_f}(t) = \begin{cases} \frac{\pi}{2} & \text{if } \sin\left(\frac{6}{10}\pi\theta_f(t)\right) > 0 \\ 0 & \text{else} \end{cases} \quad (4.41)$$

$$r_{\theta_e}(t) = \theta_e(0) - r_{\theta_f}(t).$$

The reference models parameters from Equation (4.23) are set to $\omega_n = 8$ and $\zeta = 1$. $\mathbf{Q} = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix} \Rightarrow \mathbf{P} \approx \begin{bmatrix} 3.56 & 0.078 \\ 0.078 & 0.036 \end{bmatrix}$. The adaptive gain matrices are set to $\mathbf{\Gamma}_K = \begin{bmatrix} 3 & 0 \\ 0 & 0.5 \end{bmatrix}$ and $\gamma_L = 2$. The adaptive gains are initialized to $\mathbf{K}(0) = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$ and $L(0) = 1$.

Table 4.1: Physical parameters used for the finger model.

Phalanx parameters			
Phalanx	m [kg]	l [cm]	r [mm]
Proximal Phalanx	0.020	4.8	8.5
Middle Phalanx	0.015	3.0	8.5
Distal Phalanx	0.020	2.4	8.5
Joint parameters			
Joint	k [N m rad ⁻¹]	b [N m s rad ⁻¹]	φ_{eq} [rad]
MCP Joint	0.5	0.05	$\pi/6$
PIP Joint	0.5	0.05	$\pi/4$
DIP Joint	0.5	0.05	$\pi/12$

4.3.5 Evaluation of MRAC scheme

Figure 4.8 shows a simulation of the MRAC controller over 10 s. While the MRAC scheme is active, indicated in orange, the motor angles θ_f and θ_e initially oscillate around the reference model. Over time, however, the adaptive gains $\mathbf{K}$ and L change to values that allow the system to track the reference model more closely, and the oscillations become less prominent. While the pretuned controller is active, indicated in grey, the motor tracks the reference model quite closely. Interestingly it seems to have a slightly faster response than the reference model. This might be due some inconsistencies with the cable tensioning implementation.

Continuing the simulation to 100 s, Figure 4.10 shows the evolution of the adaptive gains, aswell as the error between the motor angles and their respective reference model angles. Figure 4.9 shows the simulation results for 10 s after 90 s. Looking at Figure 4.10, after ~ 10 s, K_2 seem to converge to a certain value, while K_1 and L eventually keeps growing together, almost logarithmically. The continued growth of K_1 and L might be due to the fact that the system is trying to find gains that can track the reference model better, but is limited by the maximum current that the motor can provide. Looking at the current commands going to the motors in Figure 4.9, it can be observed that they are clipped by the maximum stall current of 4.9 A. Looking back at Figure 4.10, it's clear that the error between the motor angles and their respective reference model decrease over time as the adaptive gains update. However they don't go to zero, but end up with some steady-state error, though quite small. This tells us that the adaptive controller gains at some point reach a local optimum. Therefore, to combat the ever increasing gains, we may want to switch to a fixed-gain controller once the gains have converged.

Figure 4.11 shows a simulation over 10 s using the adaptive gains found in Figure 4.10 after 100 s, but with the gains fixed instead of adaptive. Here we can see that the

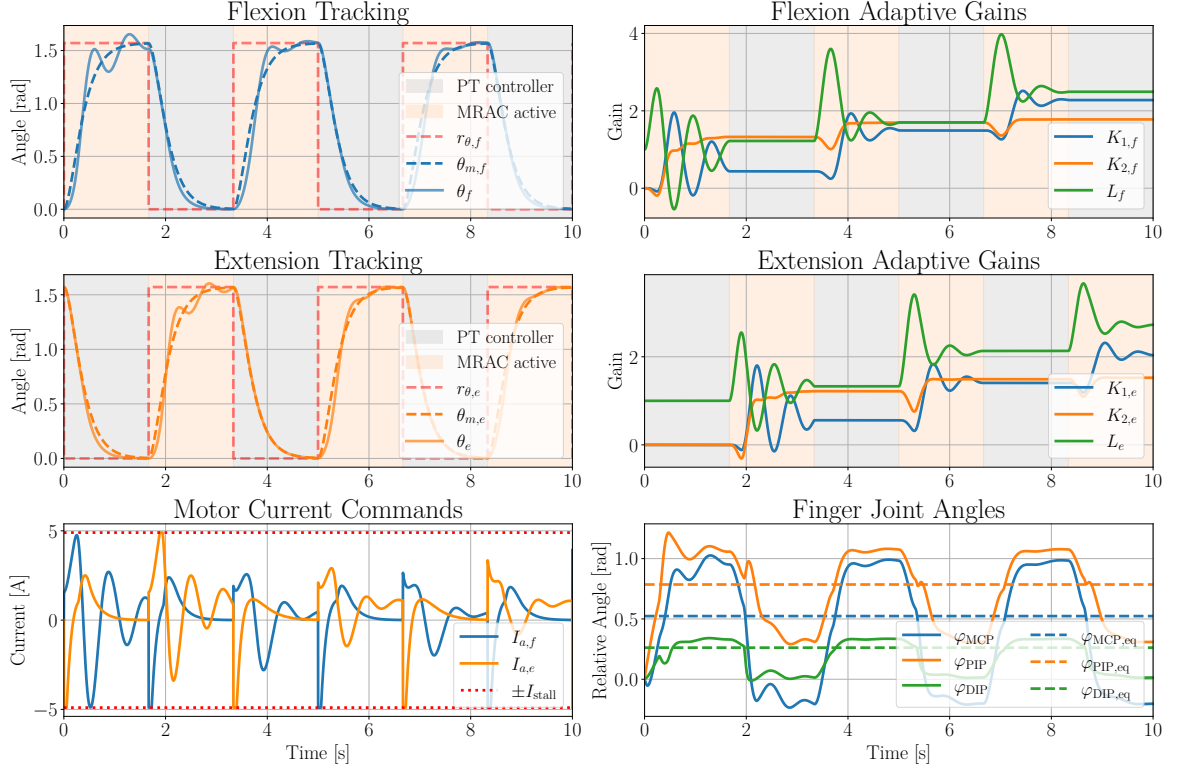


Figure 4.8: Simulation results of the MRAC scheme for flexion and extension motor over 10s.

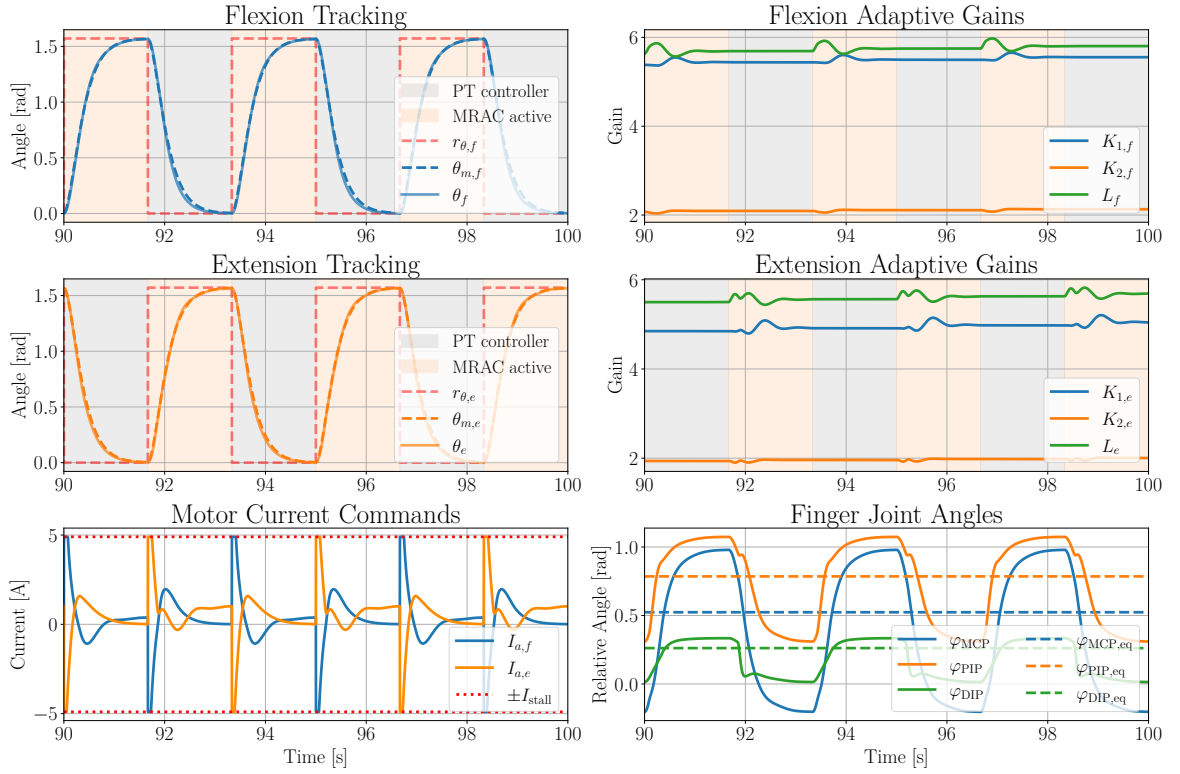


Figure 4.9: Simulation results of the MRAC scheme for flexion and extension motor after 90 s.

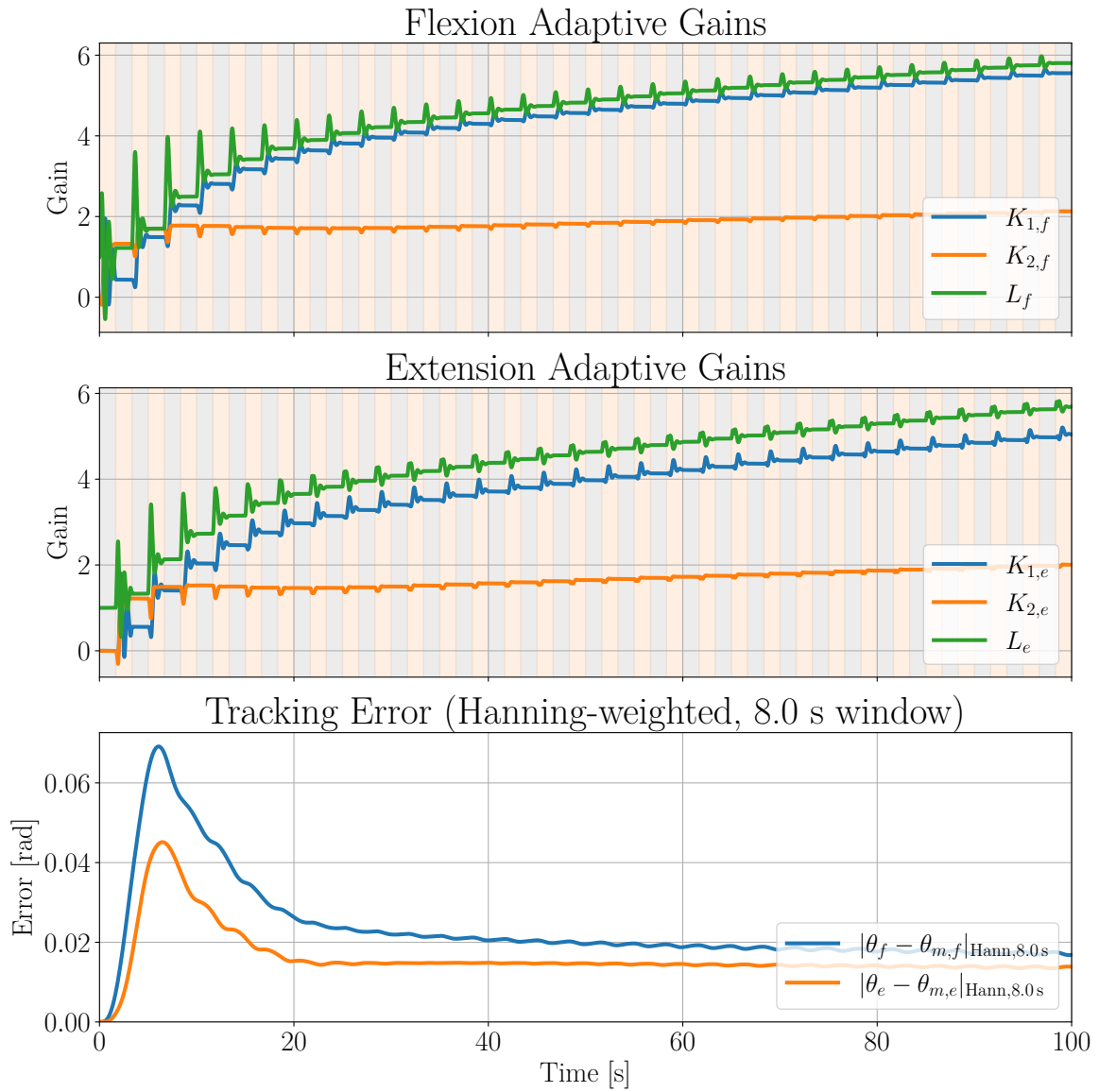


Figure 4.10: Adaptive gains and error over time for MRAC scheme for 100s, where K_1 and L keep growing together.

motors are able to track their reference angles rather well, only with a slight error in the transition to a new reference angle. Hence we may conclude that the MRAC scheme could prove useful in quickly tuning a position controller, when frequently switching between patients.

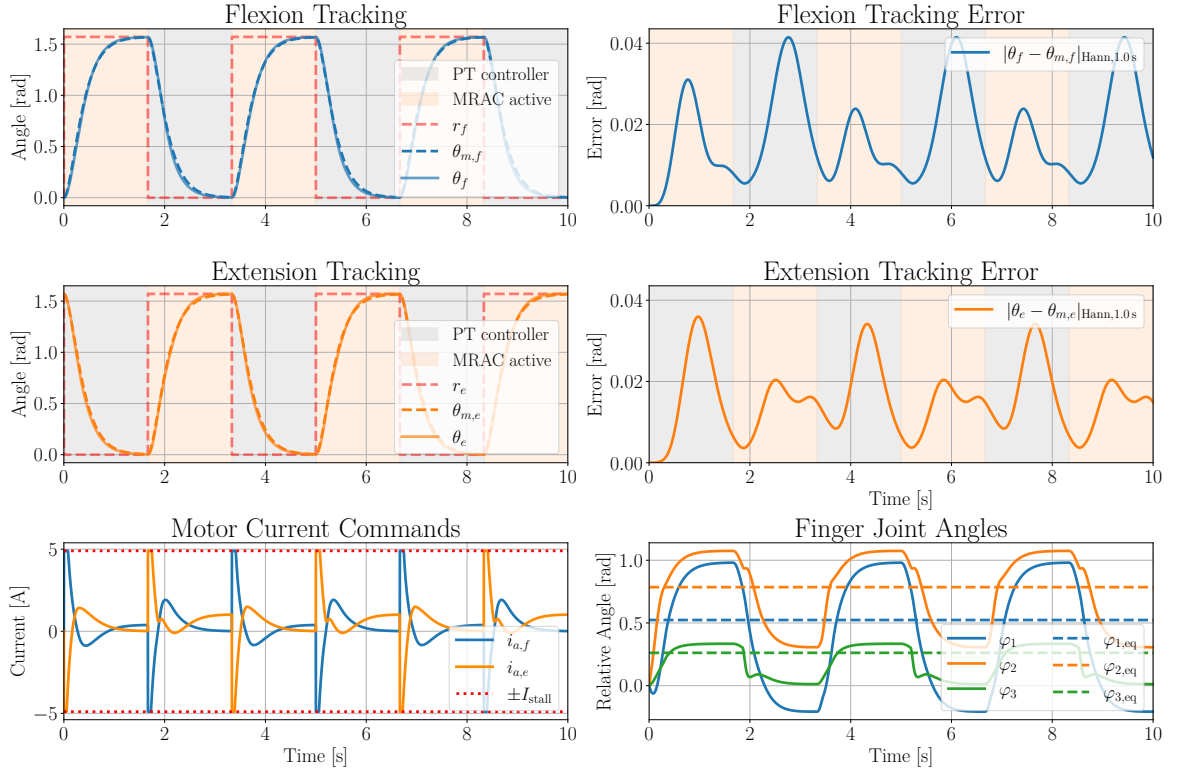


Figure 4.11: Simulation results of the MRAC scheme for flexion and extension motor over 10s, with the adaptive gains found in Figure 4.10 after 100s, but with the gains fixed instead of adaptive.

Chapter 5

Discussion

5.1 Current control

In Section 4.1, a current controller for the brushed DC motor was designed. The purpose of the controller was not only to make the armature current I_a track a reference current, but also to make it possible to treat the armature current as a control input for the slower mechanical dynamics of the motor. Considering that the generated motor torque is given by $\tau_m = K_t I_a$, controlling the armature current precisely and fast is crucial for precise torque and position control of the motor. The controller was designed as a feedforward (FF) and proportional (P) controller, shown in Equation (4.2). The feedforward term is designed to cancel out the back-EMF voltage, generated while the motor is spinning, where the proportional term is designed to stabilize the system and ensure that the armature current tracks the reference current.

Simulation showed that the feedforward term is important. Only using the proportional feedback, the controller performs reasonably well starting from rest, but the performance deteriorates as the motor speeds up. This is due to the fact that the back-EMF voltage increases linearly with the motor speed, and without the feedforward term, the controller cannot compensate for this voltage, leading to a steady-state error in the armature current. With the feedforward term included, the controller can effectively cancel out the back-EMF voltage, allowing for precise control of the armature current even at higher speeds.

Considering the fast dynamics of the armature current, the dynamics was discretized with a sampling period T_s , assuming constant angular velocity between each sample. An upper bound for the proportional gain K_p , $K_{p,\max}(T_s)$, was derived to ensure stability of the discrete-time system. Simulations showed that a gain below this upper bound resulted in a stable response, where $K_p = \frac{1}{2}K_{p,\max}(T_s)$, generally yielded a fast damped response. The sampling time showed to have a significant effect on the performance of the controller. A smaller sampling time, like $T_s = 0.2$ ms, resulted in a

fast damped response, reaching the reference current. A higher sampling time however, like $T_s = 1$ ms, resulted in an underdamped response, where the armature current got a steady-state error from the reference current. This is likely because the assumption of constant angular velocity over each sampling interval becomes less accurate as the sampling time increases. Hence the feedforward term becomes unable to account for the increasing back-EMF voltage, leading to steady-state error, like for the case with only proportional feedback. Considering that the motor inertia is unknown, and only estimated, the angular acceleration could be very different in the real system compared to the simulation. Therefore, the effect of the increasing back-EMF voltage during the sampling period could be more or less significant in the real system, compared to the simulation. The results here should hence only be seen as an indication of the effect of the sampling time, and not a precise prediction of the performance of the controller in the real system.

The noise simulations indicate that the current controller is relatively robust to current measurement noise. In general, noisy measurements would merely lead to oscillations around the steady-state value for the controller without noise present. This is an advantage of the simple structure of the controller, where no derivative actions are used, which would be very susceptible to noise.

A limitation of the current controller is that it relies on relatively accurate estimates of the parameters R_a , L_a and K_b . The parameter identification method developed in Section 3.2.2 can be used to get an estimate of the armature inductance L_a . Similar methods can also be used to estimate the armature resistance R_a and the back-EMF constant K_b . However, it remains to be seen how well this works in the real system, as there might be modelling inconsistencies throwing off the estimates.

Overall, the P+FF current controller designed in Section 4.1 shows promising results for controlling the armature current of the brushed DC motor. The simulated maximum current rate of change, approximately $\dot{I}_{a,\max} \approx 2.0 \text{ kA s}^{-1}$, is fast compared to the mechanical dynamics of the finger. It is therefore reasonable to treat the armature current as a control input in the torque and position control design.

5.2 Torque and force control

In Section 4.2, torque control was considered using the current controller from Section 4.1 as an inner loop. Since the generated motor torque is given by $\tau_m = K_t I_a$, controlling the armature current gives an indirect way to control the motor torque. The performance of the torque controller is hence highly dependent on the performance of the current controller, and the advantages and limitations of the current controller, as discussed in Section 5.1, also apply here.

As discussed in Section 4.2, there will be friction acting on the cable between the

motor and the finger, from gliding through a bowden tube. Hence, the generated force pulling on the cable from the motor, will not be equal to the applied force on the whiffle tree, and hence the finger. Therefore, without any feedback on the applied force, the applied force on the finger will generally have a steady-state error from the reference force unless the friction is compensated for. By assuming a constant friction ratio, this can be calibrated for by artificially increasing the reference force by the estimated friction force. However, this is not a very robust solution, as the friction can change over time due to wear and tear, and also depends on the configuration of the finger.

The integration of force sensors in the fingertips appears to be a more robust solution. With direct measurements of the applied force, the controller no longer has to rely solely on the current measurements as an estimate of the applied force to the finger. In Section 4.2, a feedforward (FF) and a proportional and integral (PI) controller for the reference current was designed. Simulations showed that the controller was able to reach the desired applied force in a well-damped manner, using an artificially higher reference force for the motor to compensate for the friction.

It should be noted that the performance of the FF+PI controller is dependent on the friction from the bowden tube. The proportional and integral gains were also just tuned empirically, so it is not clear how the controller would perform in the real system. The performance of the controller should hence be seen as an indication of the potential performance, rather than a precise prediction of the performance in the real system. And the controller gains should therefore be tuned carefully in the real system.

5.3 Position controller using MRAC

In Section 4.3, a position controller for the brushed DC motor was designed. The assignment called for creating an adaptive controller for fingertip positioning. Considering that there are no position sensors, IMUs, or anything alike at the hand, the position of the fingertip is unobservable. Considering that the motor is connected to the finger via a stiff cable, the position of the motor is highly coupled with the position of the fingertip. Therefore, a controller for the motor angle should prove sufficient in controlling the position of the fingertip. The position controller was hence designed as a controller for the motor angle, with the assumption that the current could be treated as a control input, using the current controller from Section 4.1 as an inner loop. The performance of the position controller is hence highly dependent on the performance of the current controller, and the advantages and limitations of the current controller, as discussed in Section 5.1, also apply here.

Considering the system is meant to be used by multiple different subjects, a pretuned controller is not guaranteed to work well for all participants. Some participants may have much stiffer joints due to little movement, and others may have more flexible

joints. Many other factors can also play a role in changing the dynamics between participants. Hence, an adaptive controller, that can adapt to the dynamics of the individual user, was designed. The adaptive controller was designed as a *model reference adaptive controller* (MRAC), where the control parameters are updated online, i.e. while the system is running, to make the output of the system follow the output of a reference model. The tracking of a reference model could also prove advantageous in providing more control over the acceleration of the gripping movement, facilitating a more comfortable user experience.

Simulations showed that the MRAC scheme was able to make the system follow the reference model rather closely over time. In the beginning the motor angles were oscillating around the reference model, but as the adaptive gains were updated, the oscillations faded and the motor angles followed the reference model closely. As discussed in Section 4.3.5, the controller gains were observed to keep growing. This is likely due to the saturation in the reference current to $I_{a, \text{stall}}$, making it a non-linearity the adaptive gains are not able to account for. We can also note that the error between the motor angles and the reference model is not going to zero, but does converge to a very small value, making it almost negligible.

The performance of the MRAC scheme can make sense considering the assumptions made in the design of the controller in Section 2.3.6. It assumes that the system is *linear time-invariant* (LTI), and that the dynamics can be written in state-space form. The model of the brushed DC motor can be written in state-space form, but the dynamics of the finger is more complex. With the cable constraint, the dynamics is hardly linear, and the equilibrium position of the finger would definitely add a constant disturbance term to the dynamics, which isn't accounted for in the MRAC derivation. In addition, the MRAC scheme, given its original assumptions, guarantees that the error between the state vector and that of the reference model goes to zero, and that the adaptive gains remain bounded. Considering the non-linearities and the saturation in the system, it is not surprising that the error doesn't go exactly to zero, and that the adaptive gains keep growing ever so slightly. However, the fact that the error converges to a very small value, and that the system is able to follow the reference model closely, indicates that the MRAC scheme is still a viable solution to position control for the fingertip positioning.

The issue of continuously growing adaptive gains can, as discussed in Section 4.3.5, be mitigated by freezing the adaptation once the tracking error between the motor angles and the reference model no longer decreases, or after a predefined adaptation period. In this way, the controller can still adapt to the dynamics of the individual user, thereby simplifying the tuning process of the position controller, where the controller can be used further with static gains rather than adaptive ones. At the same time, preventing further adaptation ensures that the controller performance does not degrade

over time as a result of unbounded growth in the adaptive gains. Simulations, using the adaptive gains found after 100 s, showed promising results. Even with static controller gains, the controller was able to follow the reference model closely. This indicates that the MRAC scheme can be used as a tool to find good controller gains for each individual user, which can then be used further with static gains. However, it remains to be seen how well this works in the real system, and whether the performance of the controller degrades over time without adaptation.

5.4 Model assumptions and limitations

In the modelling of the system, several assumptions and simplifications were made. In the modelling of the finger in Section 3.1, each phalanx was modelled as a rigid body, and the joints were modelled as ideal revolute joints, with linear torsional springs and dampers. In reality, the phalanges can't rotate freely around the joints, as they have a certain range of motion given the length of the tendons, and tissue preventing further motion. This introduces non-linearities, and also makes the stiffness of the joints non-linear, changing with the angle of the joints. Additionally the values for the model parameters used during simulations were not based on measurements from a real hand, but rather just estimated empirically to get natural looking movement. Therefore it remains to be seen how well the model captures the dynamics of the real system.

The cable connecting the motor and the finger was simplified as a constraint between the angle of the motor and the travel of the finger. It ignores any elasticity in the cable, and also doesn't model the friction from the bowden tube, which can have a significant effect on the performance of the controller, as seen with the force controller in Section 4.2.1. Additionally the length of the cable is not accurate compared to the real system, where it most likely is a lot longer. The configuration of the whiffle tree in the real system is yet to be decided, so all simulations including the whiffletree are merely based on idealized assumptions. Additionally the placement of the attachment and target points of the cables connected to the finger are not yet decided, and merely chosen empirically based on what provided good leverage for the cables pulling on the finger. The performance of the position controller in the real system might hence be very different from the simulations. Therefore it remains to be seen how well the cable constraint captures the dynamics of the real system.

Considering the assumptions and simplifications made, the performance of the MRAC scheme discussed in Section 5.3, in the real system might be very different from the simulations. The performance of the controller shown by simulations should hence only be seen as an indication of the potential performance, rather than a precise prediction of the performance in the real system. It remains to be seen how well the MRAC scheme performs in the real system, and whether it can provide a good user

experience for multiple different users. If the controller works poorly in the real system, it might be a good idea to pivot to a simple PD controller, and tune the gains for each individual user. This would likely provide better performance than a poorly performing MRAC scheme, but it would also require more time tuning the controller for each user. If there is too much friction in the system, it might also be a good idea to include an integral term in the controller, to compensate for the steady-state error from the friction. However, this would also make the tuning process more difficult, and could more easily lead to instability.

With all the assumptions and simplifications made, the performance of the controllers in the real system might be very different from the simulations. The results in this thesis should hence only be viewed as a possible proof of concept, and not a precise prediction of the performance in the real system. The controllers should be tuned carefully in the real system, and it will likely be necessary to make modifications to the controllers to get good performance in the real system.

5.5 Practical implementation considerations

Given the different controllers designed in Chapter 4, there are several practical implementation considerations to take into account when implementing the controllers in the real system. The current controller designed in Section 4.1 relies on accurate measurements of the armature current. The performance of the controller is hence highly dependent on the quality of the current sensors. Currently, the system is only equipped with a measure of the current through a pin on the motor driver. Looking at the manufacturers datasheet for the current measurement, the resolution is far too low, and the measurement is reportedly non-linear for smaller currents, making it unsuitable for use in the current controller ('Pololu - Dual MC33926 Motor Driver Carrier', 2026). Therefore, it will be necessary to add a current sensor with better resolution and that's linear for the current range of $I_a \in [-4.9, 4.9]$ A to achieve satisfactory performance from the current controller. This has been briefed with the N-Squared Lab, and they are looking into adding hall-effect current sensors in-line with the motor driver output lines. This should provide accurate measurements of the armature current, allowing for good performance from the current controller.

If the current controller is to include the feedforward term, some thought should also be put into how to get an accurate estimate of the angular velocity of the motor. The encoder used provides a digital signal with a certain resolution. Simply taking the difference between two consecutive encoder readings to get the angular velocity might not be accurate enough, especially at low speeds, or might introduce noise given the abrupt changes in the signal. Therefore, some form of filtering or estimation might be necessary to get a good estimate of the angular velocity.

An important consideration of the current controller, is that the voltage applied to the motor is given by a *pulse-width modulated* (PWM) signal. Therefore the calculated armature voltage from the current controller has to be converted to a sign, and a duty cycle, passed to the motor driver, which generates the PWM signal. The conversion from voltage to duty cycle is given by $D = \frac{|E_a|}{E_{a,\text{supply}}}$, where $E_{a,\text{supply}} = 12\text{ V}$ is the supply voltage for the motor driver. Further, the frequency of the PWM signal should be considered. A lower switching frequency will create a high-pitched noise, which can be annoying for the user. Therefore a higher frequency, preferably above hearing range, should be used. From experience working on the system, too high of a frequency can lead to the motor becoming unresponsive for lower duty cycles. Therefore a compromise between the noise and the responsiveness of the motor should be found.

As discussed in Section 5.2, the performance of the force controller is highly dependent on the friction from the bowden tube. Therefore, analysis of the friction in the real system should be done to get a better understanding of how it affects the performance of the controller, and to get a better idea of how to tune the controller for good performance. If the integration of force sensors in the fingertips is to be implemented, insight to the friction could also be useful for updating the feedforward term, which can facilitate for a better performance from the controller.

For safety reasons, the code should implement failsafe mechanisms to prevent the motors from applying too much force, which could potentially harm the user. This can be done by implementing limits on the reference current, or by shutting down the system if the measured current exceeds a certain threshold. Startup of the motors generally results in a large inrush current, which could potentially trigger the failsafe mechanism. This should be kept in mind for the implementation of the failsafe mechanism, to prevent it from being triggered unnecessarily. If the final system includes the integrated force sensors, these should likely be used here as well to provide further insight into what the user is experiencing. Some kind of physical emergency stop button should also be implemented, to allow the user and or operator to quickly shut down the system if something goes wrong.

5.6 Future work

Considering all the controllers designed in Chapter 4 are only evaluated through simulations, the next step would be to implement the controllers in the real system and evaluate their performance. The performance of the controllers in the real system might be very different from the simulations, due to the assumptions and simplifications made in the modelling of the system, as discussed in Section 5.4. Therefore, it's important that this tuning takes place prior to running trials on participants. The tuning of the current controller should be done first, as the performance of the torque and position

controllers is highly dependent on the performance of the current controller. This can be done with an unloaded motor, and includes first estimating the armature inductance L_a and rotor inertia J_m as described in Section 3.2.2, and secondly setting K_p in the proportional controller to some fraction of $K_{p,\max}(T_s)$ based on the delay T_s in the control loop of the microcontroller. For the torque and position controllers, the motors has to be loaded. For the torque controller, this can simply be done by attaching the cable to a fixed point. For the position controller however, the cable should be connected through a whiffle tree to a model hand, to provide mostly similar dynamics of the finger for the MRAC scheme. The N-Squared Lab has access to such a model hand. The hand should not be interacting with the environment during the tuning of the position controller, to prevent unnecessary disturbances to the MRAC scheme. Most likely, tuning of the adaptive gain matrices $\mathbf{\Gamma}_K$ and γ_L , the initial values of the adaptive gains, and $\mathbf{Q}$ from Section 4.3, will be necessary to get good performance from the MRAC scheme. ζ and ω_n from Equation (4.23) should also be selected based on the desired response of the system. Additionally, the MRAC scheme should be run with an unloaded motor to find the constant gains to be used when the motors are detensioning the cable.

The controllers are only designed for one finger. Considering that the system is meant to control 6-DOF, the controllers will need to be copied, and implemented for all fingers. Hence the tuning process above will have to be repeated for each degree of freedom. The performance of the controllers when running simultaneously for all fingers should also be evaluated, to ensure that there are no issues with the controllers interfering with each other.

Provided that the current controller performs as intended, the force sensors currently mounted on the system will likely become redundant. Considering these sensors measure the cable force before the Bowden tube, they provide little to no additional information beyond what is already entailed by the armature current. Hence, the force sensors could be removed to save cost and reduce complexity. Additionally, the cables going from the motors could then be more in line with the hand, further reducing the friction in the system. The addition of the integrated force sensors in the fingertips, as discussed in Section 4.2.1, would however likely provide a significant improvement in the performance of the force controller, and should therefore be considered for future iterations of the system.

The performance of the MRAC scheme for position control should also be evaluated in the real system, and compared to a simple PD controller to see if the added complexity of the MRAC scheme is justified by a significant improvement in performance.

A major aspect of the system yet to be considered in this thesis, is the switching between position and force control. For a good user experience, the system should be able to switch seamlessly between position and force control, depending on the task at

hand. Without the integrated force sensors in the fingertips, this could prove difficult, as the system then would have to estimate contact with the environment based on the armature current. Generally, the armature current would increase when the finger comes in contact with the environment, as the motor has to apply more torque to reach the desired position, which can be used as an indication of contact. A much more robust solution would be to use the integrated force sensors in the fingertips. With them, the system can start out with position control, to close the hand, and when the force sensors detect contact with the environment, the system can seamlessly switch to force control to apply the desired force on the environment. It's important that this switching is evaluated carefully, to ensure that it provides a good user experience.

Chapter 6

Conclusion

This thesis has investigated modelling, parameter estimation, and control strategies for a tendon-driven hand exoskeleton intended to control both fingertip position and interaction forces. The system is intended for use in spinal-cord-injured patients, with the goal of improving hand function and independence. The main challenges addressed include the unknown and user-dependent dynamics of the fingers, the nonlinearities and friction in the Bowden-tube cable transmission, and the limited measurements at the hand. The work has focused on the design and simulation of control strategies, with the goal of providing a framework for future implementation and testing on the physical system. This includes modeling of a brushed DC motor, a planar finger, the cable transmission between the motors and the finger, as well as the design of a discrete-time current controller, a torque control scheme, and a model reference adaptive position controller.

A model of the brushed DC motor was obtained, including the electrical dynamics of the armature circuit and the mechanical dynamics of the rotor. Estimates for the motor parameters were calculated from the motor datasheet, and a method for estimating the remaining unknown parameters was developed and simulated. Further, a simplified planar model of the finger was derived using the Euler-Lagrange equations, including the effects of joint stiffness and damping. The parameters of the finger model were estimated empirically through simulations, to provide a natural looking motion of the finger. The cable connecting the motor to the finger was assumed to be sufficiently stiff, and modeled as a constraint relating the motor angle to the finger joint angles.

A discrete-time current controller was designed for the brushed DC motor, using proportional feedback with a feedforward term to compensate for the back-EMF. Simulations showed that the controller was able to track the reference current very rapidly, compared with the slow mechanical dynamics of the motor. This supports the goal of using the armature current as a control input for both torque and position control. The stability of the discrete-time current controller was analyzed, and resulted in an upper bound for the proportional gain ensuring stability, given the controller

sampling time and some of the motor parameters. Simulations confirmed that the controller remained stable for gains below the upper bound. They also showed that a short sampling time is important for achieving good current tracking, where a longer sampling time lead to steady-state errors and a slower response. Given the simplicity of the controller, simulations also showed it being robust to measurement noise.

Torque and force control were then considered using the current controller as its basis. Due to the generated motor torque being proportional to the current, controlling the current provided a indirect way to control the torque output of the motor. Considering the friction acting on the cable between the motor and the finger, the applied force at the fingertip won't be equal that which the motor pulls on the cable. Hence, integrated force sensors in the fingertips was considered. Using a proportional and integral feedback with a feedforward term, the fingertip force sensors were shown through simulations to be able to combat the effects of friction and improve the tracking of the desired fingertip force, and should hence be considered for the real system.

For position control, given that the system is meant to be used by several different subjects, with different finger stiffness, retuning of the controller for each user would likely be required to provide a good user experience. To address this and to hopefully make the tuning process easier, a model reference adaptive control scheme was considered. Since the position of the fingertip is not directly measured, the motor angle was used as a proxy for the finger position, based on the assumption of a sufficiently stiff cable connection between the motor and the finger. Simulations showed that the MRAC scheme was able to make the motor angle track a reference model well over time. Initially it showed oscillatory behavior about the reference model. But over time, as the adaptive gains updated, these oscillations dissapeared, and the controller was able to track the reference model very closely. However, the adaptive gains were also observed to continue growing slowly over time, and the error between the motor angle and the reference model never converged exactly to zero. This is most likely due to nonlinearities from the cable constraint, saturation in the armature reference current, and model structures that violate the ideal assumptions of the MRAC derivation. In practice, this signals that the MRAC scheme is to be rather used as an automatic tuning method for finding good fixed gains. That way the controller can freeze the adaptation gains after the error stops decreasing, or a after a predefined time period. This was tested in simulations, and showed that the controller was able to track the reference model well, even with static gains found by the MRAC scheme.

Considering that the controllers developed doesn't rely on any force measurements prior to the bowden tube, the force sensors currently mounted on the system can be considered redundant, and could be removed in future iterations of the system. This would reduce the cost and complexity of the system, and possibly reduce overall friction in the system.

As the results in this thesis are merely based on simulations, they should be interpreted as simulation-based proof of concepts rather than a validation of the final system. Several important effects were simplified or omitted, including nonlinear joint stiffness, how the tendons are routed, cable elasticity, detailed Bowden-tube friction, exact whiffle-tree geometry, and likely many other factors. Hence, careful implementation and testing on the physical system will be required to validate the results, and to further refine the models and control systems.

Bibliography

- ICS Glossary*. (2026, May 24). ICS. <https://www.ics.org/glossary/investigation/electromyographyemgintramuscular>
- Ioannou, P., & Fidan, B. (2006, January 1). *Adaptive Control Tutorial*. SIAM.
- Ioannou, P. A., & Sun, J. (2012, December 19). *Robust Adaptive Control*. Courier Corporation.
- Jeong, U., & Cho, K.-J. (2015). Feedforward friction compensation of Bowden-cable transmission via loop routing. *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5948–5953. <https://doi.org/10.1109/IROS.2015.7354223>
- National Cancer Institute. (2026). *Anatomical Terminology | SEER Training*. Retrieved March 13, 2026, from <https://training.seer.cancer.gov/anatomy/body/terminology.html>
- Planar Manipulator - an overview | ScienceDirect Topics*. (2026, May 24). <https://www.sciencedirect.com/topics/engineering/planar-manipulator>
- Pololu - 75:1 Metal Gearmotor 25Dx54L mm HP 12V*. (2026, May 15). <https://www.pololu.com/product/3206/resources>
- Pololu - Dual MC33926 Motor Driver Carrier*. (2026, May 23). <https://www.pololu.com/product/1213#:~:text=motor%20driver%20chips,-Current%20sensing,-The%20current%20sense>
- Pololu 25D Metal Gearmotors. (2021). <https://www.pololu.com/file/0J1829/pololu-25d-metal-garmotors.pdf>
- Rahman, N., & Mat Yahya, N. (2021). A mathematical model of a brushed DC motor system. *Data Analytics and Applied Mathematics (DAAM)*, 2, 60–68. <https://doi.org/10.15282/daam.v2i2.6830>
- TeachMeAnatomy. (2026a). *Anatomical terms of location*. Retrieved March 14, 2026, from <https://teachmeanatomy.info/the-basics/anatomical-terminology/terms-of-location/>
- TeachMeAnatomy. (2026b). *Anatomical terms of movement*. Retrieved March 15, 2026, from <https://teachmeanatomy.info/the-basics/anatomical-terminology/terms-of-movement/>

- Ting, J., Del Vecchio, A., Friedenberg, D., Liu, M., Schoenewald, C., Sarma, D., Collinger, J., Colachis, S., Sharma, G., Farina, D., & Weber, D. J. (2019). A wearable neural interface for detecting and decoding attempted hand movements in a person with tetraplegia. *2019 41st Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 1930–1933. <https://doi.org/10.1109/EMBC.2019.8856483>